miniengine

API-referentie

HTTP, websocket en SDK voor Mini Engine 0.6: kamers, rollen, acties, realtime, achievements, eigen sets, accounts en het uploaden van eigen games. Openbare editie.

Versie 0.6, oktober 2026. Openbare editie. Routes onder `/api/v1`.

Inhoud

Zo lees je dit 3
- Nieuw sinds 0.5 3

Begrippen 5
- Rollen en wat ze zien 5
- Hostmodi en runtimes 6
- Versies en action-ID's 6

Authenticatie 7

Routeoverzicht 9

HTTP: games 11

HTTP: kamers 15
- Maken, deelnemen en lezen 16
- Spelverloop 19
- Kamerbeheer 21

HTTP: accounts 22
- Robotcheck 24
- E-mailbevestiging 24

Ontwikkelaars 26
- Mijn spellen op de accountpagina 28
- Een versie uploaden 29

Versies en kamers 32

Websocket 34

Realtime 37

Achievements 40
- Toekennen vanuit een eigen game 41

Eigen sets 44
- Het set-schema 44
- Een kamer met een eigen set 46

Ingebouwde serveracties 48

SDK-client 49
- client.realtime 52

Manifest en instellingen 54

Grenzen en rate limits 57

Foutformaat en foutcodes 60

Zo lees je dit

Deze referentie beschrijft de publieke interface van Mini Engine 0.6: de HTTP-routes, het websocketprotocol met realtime, de TypeScript-SDK, het gamepackage en het uploaden van eigen games. Dit is de openbare editie: alles wat een client, een game of de `me`-CLI gebruikt. Hoe je de server zelf draait of beheert, staat er niet in.

Nieuw sinds 0.5

- **De host speelt mee (0.5.1).** Een browsergame (`runtime:"host"`) mag `playing` gebruiken. De stoel van een meespelende host telt altijd als klaar, elke snapshot heeft `host_player_id`, en de host kan een achievement aan zijn eigen stoel geven. Zie *Hostmodi en runtimes*.
- **Iedereen doet mee, of TV (0.5.2).** Een browsergame mag ook `tv` aanbieden: het apparaat dat de kamer maakt, is de tv, toont het spel en heeft geen stoel. In het formulier heet `playing` *Iedereen doet mee*. Gezelschapsspellen bieden `["playing", "tv"]` met `playing` als standaard. In een `tv`-kamer kan de eigenaar van de tv op de eigen telefoon achievements krijgen.
- **Een eigen icoon (0.5.1).** Het manifest mag `icon` hebben: een SVG in `assets/`. Het staat in `GET /api/v1/games` en op de spelkaarten. Zie *Manifest en instellingen*.
- **Mijn spellen (0.5.1).** Met `GET /api/v1/accounts/games` en `PUT /api/v1/accounts/games/{id}` zet een ontwikkelaar zijn eigen spellen aan of uit met zijn gewone accountsessie, zonder dev-key.
- **Eigen sets als formulier (0.5.3).** Een soort set met platte velden krijgt een formulier: een lijst met teksten is een tekstvak met één tekst per regel, een lijst van kleine items een blok per item. Zie *Eigen sets*.
- **De web-app (0.6).** Spellen staan als kaarten: het icoon groot op de eigen kleur van het spel, met het aantal spelers, tv, eigen sets en achievements. Meedoen gebeurt met een code in zes tegels. De pagina met alle spellen heet nu `/home/games` (was `/home/spellen`), en na het bewaren van een set opent `/home/sets/{id}?saved` (was `?opgeslagen`). Het formulier voor een nieuwe kamer toont de instellingen in de volgorde van `required`.
- **SDK 0.5.2** met `'tv'` in `HostMode`, `icon` in `Manifest` en `host_player_id` in `Snapshot`. Packages en kamers uit 0.5 werken ongewijzigd.

Basisadres en versies

Het basisadres is dat van de server, zoals `https://miniengine.nl`; met `me dev` uit de CLI is het `http://127.0.0.1:3170`. Alle API-routes staan onder `/api/v1`; dat is de routeversie, los van de versie van Mini Engine. Alleen `/api/health` staat daarbuiten. Voorbeelden gebruiken relatieve paden; zet er je eigen basisadres voor.

Verzoeken en antwoorden

- Verzoeken met een body sturen JSON met `Content-Type: application/json`. Onbekende velden worden afgewezen.
- Antwoorden zijn JSON, behalve de HTML-pagina's, de gameassets, de releaseassets en routes die `204 No Content` geven.

- Elke fout heeft dezelfde vorm: `{"error":{"code":"…","message":"…"}}`. De `code` is stabiel; de `message` is Nederlands en bedoeld voor mensen.
- Kamercodes zijn niet hoofdlettergevoelig. `tafels` en `TAFELS` zijn dezelfde kamer.

Notatie

Notatie	Betekenis
`{code}`	Een padparameter, hier de kamercode van zes letters.
`HOST_SECRET`	Een plaatsvervanger voor een geheim token, zoals ook `PLAYER_SECRET` en `ACCOUNT_SECRET`. Een dev-key staat er als `med_…`. Echte tokens staan nooit in dit document.
`…`	Ingekort voorbeeld. Het echte antwoord bevat meer velden of langere waarden.
Toegang	Welke sleutel de route verwacht: openbaar, een kamertoken van een bepaalde rol, een accounttoken of een dev-key.

Vertrouwen. Ingebouwde en geüploade games draaien als vertrouwde code op dezelfde origin als Mini Engine. Ze kunnen in de browser de kamertoken van dat tabblad en een eventuele accountsessie lezen, en daarmee ook de accountroutes aanroepen, zoals de sets van die speler. Daarom krijgen alleen uitgenodigde ontwikkelaars een dev-key. Een aparte origin per game bestaat in 0.6 niet.

Begrippen

Eén kamer, één spel, een handvol apparaten. De server is het middelpunt: hij bewaart de stand, controleert wie wat mag en stuurt iedere rol alleen wat die rol mag zien.

Kamers

Een kamer hoort bij precies één game. Hij heeft een code van zes hoofdletters uit `ABCDEFGHJKLMNPQRSTUVWXYZ` (geen I en O, geen cijfers) en een status die de levenscyclus volgt:

```
lobby → running → finished → lobby
```

Nieuwe spelers komen in de lobby binnen, of tijdens `running` als late joining is toegestaan. Terug naar de lobby geeft een nieuwe `round_id`, zet readiness en spelstate terug en begint een nieuwe deduplicatieperiode. Een kamer verloopt na twaalf uur zonder activiteit; verbonden apparaten houden hem levend. Kamers, accounts, resultaten, achievements en sets worden opgeslagen en overleven een herstart van de server. Aanwezigheid begint na een herstart als offline.

Naast die opgeslagen state heeft elke kamer een realtimekanaal voor snelle events en live waarden, zoals een buzzer of de positie van een kart. Realtime staat alleen in het geheugen en raakt de state en de versie nooit; zie *Realtime*.

Ingebouwde en geüploade games

Ingebouwde games, zoals Common Dice en Jeopardy, horen bij de server zelf. Vanaf 0.3 kunnen uitgenodigde ontwikkelaars ook eigen browserhost-games uploaden met een dev-key; aan- of uitzetten kan sinds 0.5.1 ook op de accountpagina. Elke upload is een vaste versie (een *release*); één versie per game is actief en die krijgen nieuwe kamers. Een kamer houdt de versie waarmee hij is gemaakt. Zie *Ontwikkelaars* en *Versies en kamers*.

Rollen en wat ze zien

Elk kamertoken hoort bij één rol. De rol bepaalt welke delen van de state in de snapshot staan.

Rol	Wie	Ziet
`host`	De maker van de kamer. Blijft `host`, ook als hij meespeelt.	`public_state`, `host_state`, alle `player_states`; als hij meespeelt ook zijn eigen `private_state`.
`player`	Iemand die met een naam is binnengekomen.	`public_state` en alleen zijn eigen `private_state`.
`display`	Het gedeelde scherm, bijvoorbeeld de tv.	Alleen `public_state`. Nooit private state.

Namen, aanwezigheid, readiness, eventuele account-ID's, instellingen, de kamerconfig en de naam van een gekozen eigen set zijn voor alle rollen zichtbaar. De openbare info-route toont geen namen en geen spelstate.

Hostmodi en runtimes

De hostmodus ligt vast bij het maken van de kamer. De game bepaalt welke modi mogen.

Hostmodus	De maker	Spelverloop
`moderator`	Host zonder spelersstoel; `player_id` is `null`.	Een aparte spelleider bedient het spel en heeft zelf geen readiness.
`playing`	Host met spelersstoel.	*Iedereen doet mee*: de maker speelt mee en start het spel.
`tv`	Host zonder spelersstoel; `player_id` is `null`.	*TV* (sinds 0.5.2): het apparaat dat de kamer maakt, toont het spel en speelt niet mee. Alleen bij `runtime:"host"`.
`hostless`	Host met spelersstoel.	Deelnemers bedienen het servergame; het tabblad van de maker hoeft niet verbonden te blijven.

De **runtime** zegt waar de spelregels draaien.

- `runtime:"server"`: de regels draaien op de server. Dat geldt alleen voor de ingebouwde games Common Dice (alle drie de modi, standaard `hostless`) en Jeopardy (alleen `moderator`). Een manifestveld alleen installeert geen nieuwe serverruntime.
- `runtime:"host"`: een vertrouwde browserhost verwerkt acties en publiceert de volledige state. Toegestaan zijn `playing`, `tv` en `moderator`, in elke combinatie; `hostless` kan niet, want het hosttabblad moet verbonden blijven voor de spelregels.

Een gezelschapsspel (bord, kaart, dobbel, tekenen) biedt `playing` en `tv`, met `playing` als standaard: iedereen doet mee, of een tv toont het spel. Alleen een spel met een neutrale leider, zoals een quiz, kiest `moderator`. Een meespelende host in een browsergame:

- heeft een eigen stoel: `player_id` in zijn credentials, met de naam uit `name` bij het maken. Elke snapshot noemt die stoel in `host_player_id`; zonder meespelende host is dat `null`.
- staat altijd klaar. De engine zet zijn stoel op klaar bij het maken en na elke nieuwe lobby, en die stoel houdt de start nooit tegen.
- stuurt geen acties. De hostpagina is de scheidsrechter en past de eigen zetten zelf toe met `setState`.
- telt mee voor `min_players` en `max_players`, kan niet verwijderd worden en kan de kamer niet verlaten.

Versies en action-ID's

Elke kamer heeft een `version`. Gameplay, instellingen, config en lifecycle kunnen die verhogen. Aanwezigheid, binnenkomen, vertrekken en readiness geven ook een snapshot, maar verhogen de versie niet. Filter snapshots dus niet weg alleen omdat de versie gelijk bleef.

Wijzigingen die een gelijktijdige schrijver kunnen overschrijven, sturen `expected_version` mee: state publiceren, instellingen vervangen en de config aanpassen. Klopt de versie niet meer, dan volgt `409 version_conflict`. Lees de nieuwste snapshot en beoordeel de handeling opnieuw.

Acties op een servergame mogen een `action_id` meekrijgen: 1–64 tekens zonder controletekens. De server onthoudt per kamer de laatste 256 verwerkte ID's, gekoppeld aan actor en ronde. Een herhaling binnen dat venster slaagt zonder tweede effect en zonder versieverhoging. De SDK maakt standaard een unieke ID per actie en herhaalt opdrachten nooit vanzelf.

Authenticatie

Er zijn drie soorten sleutels die los van elkaar staan. Gebruik ze nooit door elkaar. Alle drie gaan als bearer in de header, nooit in de URL of een logregel.

```
Authorization: Bearer <token>
```

Sleutel	Krijg je van	Gebruik je voor
Kamertoken	Kamer maken (host), deelnemen (speler) of een displaytoken opvragen (scherm)	Alles binnen één kamer
Accounttoken	Registreren of inloggen	`/api/v1/accounts/…`, en optioneel bij kamer maken (verplicht met `set_id`) en deelnemen
Dev-key (`med_…`)	Wie de server beheert	Je eigen games uploaden en beheren onder `/api/v1/dev`

Van elke sleutel bewaart de server alleen een hash: een kwijtgeraakte sleutel is niet terug te halen.

Kamertokens

Maken en deelnemen geven een token van 64 tekens terug, met de bijbehorende rol. De server bewaart alleen een SHA-256-hash. Er is één token voor de host, één per speler en één voor het gedeelde scherm.

- Eén token heeft één actieve websocketverbinding. Een nieuwe verbinding met hetzelfde token vervangt de oude; die krijgt `session_replaced`.
- Vertrekken of verwijderd worden trekt het spelerstoken in. Een kamer verwijderen trekt alles in.
- De SDK bewaart kamertokens in `sessionStorage` onder `mini-engine:CODE:ROLE`.

Displaytokens

Het token voor het gedeelde scherm zit in het antwoord van `POST /api/v1/rooms` als `display_token`, tenzij het scherm is uitgeschakeld. Het scherm opent `/room/CODE/display#TOKEN`: de schil bewaart het fragment en haalt het weg voor het gameframe laadt, dat alleen `?room=CODE` krijgt.

De host kan een nieuw displaytoken opvragen; het vorige werkt dan niet meer. Het scherm uitschakelen trekt het token blijvend in. Na opnieuw inschakelen vraagt de host een nieuw token op; het oude komt niet terug.

Accounttokens

Accounts zijn optioneel; gastspel werkt zonder. Registreren en inloggen geven een willekeurig sessietoken van 64 tekens dat dertig dagen geldig is en een herstart overleeft. Elke login maakt een eigen sessie; uitloggen trekt alleen de gebruikte sessie in.

Stuur een accounttoken mee bij het maken of binnenkomen van een kamer om de spelersstoel aan het account te koppelen. De server controleert het token; een ongeldig token geeft `401`. De body kan geen `account_id` opgeven. Eén account bezet hoogstens één actieve stoel per kamer.

Bij het maken onthoudt de kamer ook het account van de maker, in elke hostmodus. Dat bepaalt of de host achievements mag toekennen (zie *Achievements*). Een eigen set kiezen met `set_id` kan alleen met een accounttoken.

Sinds 0.5.1 zet een ontwikkelaar zijn eigen spellen ook met zijn accounttoken aan of uit, onder `/api/v1/accounts/games`. Daar is geen dev-key voor nodig.

Dev-keys

Een dev-key hoort bij een gewoon account met een bevestigd e-mailadres en geeft toegang tot de routes onder `/api/v1/dev`. Je krijgt hem van wie de server beheert. De key is `med_` gevolgd door 64 kleine hexadecimale tekens. Je ziet hem één keer; de server bewaart alleen een hash.

- Een key kan een verloopdatum hebben (1–3650 dagen) of onbeperkt geldig zijn.
- Een ingetrokken key werkt meteen niet meer. Games die al zijn geüpload blijven staan.
- Een ongeldige, verlopen of ingetrokken key geeft `401 unauthorized`. Elk gebruik werkt `last_used_at` bij.

Origin

Verzoeken met een `Origin`-header worden alleen geaccepteerd als die origin gelijk is aan de host van het verzoek, of als de server haar uitdrukkelijk toestaat. Anders volgt `403 origin_forbidden`. Verzoeken zonder `Origin`, zoals uit Node of `curl`, gaan gewoon door. Elk antwoord krijgt `no-store`, `nosniff`, `no-referrer` en een strikte content security policy die alleen `'self'` toestaat.

Routeoverzicht

Alle routes die een client, een game of de `me`-CLI gebruikt, op één plek. Sinds 0.5.1 is er `/api/v1/accounts/games`. De details staan per groep in de hoofdstukken hierna.

Methode	Pad	Toegang	Resultaat
GET	`/api/health`	openbaar	Status en engineversie
GET	`/api/v1/games`	openbaar	Alle speelbare games
GET	`/api/v1/games/{id}`	openbaar	Metadata van één game
GET	`/api/v1/games/{id}/{role}`	openbaar	HTML voor `host`, `player` of `display`; met `?room=` de versie van die kamer
GET	`/games/{id}/assets/{path}`	openbaar	Publieke assets van de actieve versie
GET	`/releases/{id}/{version}/assets/{path}`	openbaar	Assets van de actieve versie, of van een versie die een lopende kamer speelt
POST	`/api/v1/rooms`	gast of account	201 met hosttoken; met `set_id` alleen met account
POST	`/api/v1/rooms/{code}/join`	gast of account	201 met spelerstoken
GET	`/api/v1/rooms/{code}/info`	openbaar	Openbare kamerinfo
GET	`/api/v1/rooms/{code}`	kamertoken	Snapshot voor jouw rol
GET	`/api/v1/rooms/{code}/game-data`	host	De eigen set van de kamer, anders de private `data.json`
POST	`/api/v1/rooms/{code}/ready`	deelnemer, lobby	Readiness zetten
POST	`/api/v1/rooms/{code}/actions`	speler; host bij servergame	Actie sturen
PUT	`/api/v1/rooms/{code}/state`	host, hostruntime	Volledige state vervangen
PUT	`/api/v1/rooms/{code}/settings`	host	Instellingen vervangen
POST	`/api/v1/rooms/{code}/lifecycle`	host	Status doorzetten
PUT	`/api/v1/rooms/{code}/config`	host, lobby	Kameropties aanpassen
POST	`/api/v1/rooms/{code}/display-token`	host, scherm aan	Nieuw displaytoken
POST	`/api/v1/rooms/{code}/achievements`	host, hostruntime	Achievement toekennen
POST	`/api/v1/rooms/{code}/leave`	speler	204, token ingetrokken
DELETE	`/api/v1/rooms/{code}/players/{player_id}`	host	204, speler verwijderd
DELETE	`/api/v1/rooms/{code}`	host	204, kamer verwijderd
GET	`/api/v1/rooms/{code}/ws`	kamertoken in eerste bericht	Websocket

Methode	Pad	Toegang	Resultaat
GET	/api/v1/accounts/config	openbaar	Sitekey voor de robotcheck
POST	/api/v1/accounts/register	openbaar	201 met account en token; bevestigingsmail
POST	/api/v1/accounts/login	openbaar	200 met account en token
GET	/api/v1/accounts/me	account	Accountobject
POST	/api/v1/accounts/logout	account	204, sessie ingetrokken
GET	/api/v1/accounts/stats	account	Statistieken
POST	/api/v1/accounts/verify-email	account van de link	E-mailadres bevestigen
PUT	/api/v1/accounts/email	account	Adres zetten of link opnieuw sturen
GET	/api/v1/accounts/achievements	account	Achievements van het account
GET	/api/v1/accounts/sets	account	Jouw sets, zonder inhoud
POST	/api/v1/accounts/sets	account	201, set bewaard
GET	/api/v1/accounts/sets/{id}	account	Eén set met inhoud
PUT	/api/v1/accounts/sets/{id}	account	Titel en inhoud vervangen
DELETE	/api/v1/accounts/sets/{id}	account	204, set verwijderd
GET	/api/v1/accounts/games	account	Jouw geüploade games
PUT	/api/v1/accounts/games/{id}	account	Game aan- of uitzetten
GET	/api/v1/dev/me	dev-key	Account en key
GET	/api/v1/dev/games	dev-key	Jouw games
GET	/api/v1/dev/games/{id}	dev-key	Eén game
PUT	/api/v1/dev/games/{id}	dev-key	Game aan- of uitzetten
DELETE	/api/v1/dev/games/{id}	dev-key	204, game met alle versies weg
POST	/api/v1/dev/games/{id}/versions	dev-key	201, nieuwe versie geüpload
POST	/api/v1/dev/games/{id}/versions/{version}/activate	dev-key	Versie actief maken
DELETE	/api/v1/dev/games/{id}/versions/{version}	dev-key	204, versie verwijderd

Daarnaast serveert de engine de pagina's onder `/home`, de kamerschil `/room/{code}/{role}` en de SDK onder `/sdk/`; zie *HTTP: games*. Oude adressen onder `/demo` verwijzen door naar `/home`.

HTTP: games

Openbare routes die de geïnstalleerde games en hun bestanden beschrijven. Geen token nodig.

`GET /api/health`

Toegang openbaar

Status en de gebouwde engineversie.

Antwoord 200

```
{"status": "ok", "version": "0.6.1"}
```

`GET` `/api/v1/games`

Toegang openbaar

Alle speelbare games met metadata en instellingenschema: de ingebouwde games en de aangezette uploads, elk met de metadata van de actieve versie. Uitgezette en geblokkeerde uploads staan er niet in. `achievements` en `set_kinds` staan er altijd in, zo nodig als lege lijst. `icon` staat er alleen in als het manifest het declareert, precies zoals daar geschreven, bijvoorbeeld `"icon": "assets/icon.svg"`. Laad het als `/games/{id}/{icon}?v={version}`. De velden staan bij *Manifest en instellingen*.

Antwoord 200


```
{
  "games": [
    {
      "id": "common-dice",
      "title": "Common Dice",
      "description": "Eén worp voor de hele tafel. …",
      "version": "2.0.0",
      "min_players": 1,
      "max_players": 100,
      "has_display": true,
      "supports_late_join": true,
      "default_settings": {
        "dice_count": 5, "sides": 6, "roll_permission": "everyone", "theme": "ivory"
      },
      "settings_schema": {
        "type": "object", "properties": {…}, "required": […], "additionalProperties": false
      },
      "runtime": "server",
      "host_modes": ["moderator", "playing", "hostless"],
      "default_host_mode": "hostless",
      "achievements": [
        {"id": "dubbel-zes", "title": "Dubbel zes",
         "description": "Gooi minstens twee zessen in één worp."},
        {"id": "hoogste-worp", "title": "Hoogste worp", "description": "…"}
      ],
      "set_kinds": []
    }
  ]
}
```


`GET` `/api/v1/games/{id}`

Toegang openbaar

De metadata van één game, in dezelfde vorm als een item uit de lijst.

Fouten: `404 game_not_found` (ook voor een uitgezette of geblokkeerde upload)

`GET /api/v1/games/{id}/{role}`

Toegang openbaar

De HTML-pagina van de game voor `host`, `player` of `display`, als `text/html`. De launcher laadt deze pagina in het gameframe.

Met `?room=CODE` serveert de engine de HTML van de versie waarmee die kamer is gemaakt. Bij een geüploade game verwijst hij daarin `/games/{id}/assets/` naar `/releases/{id}/{version}/assets/`. Zonder `?room=` krijg je de actieve versie van een speelbare game.

Fouten: `404 game_not_found` (onbekend, geblokkeerd, of uitgezet en zonder kamer), `404 game_page_not_found`, `404 release_not_found`

`GET /games/{id}/assets/{path}`

Toegang openbaar

Een publiek bestand uit de map `assets/` van de actieve versie van de game, met een MIME-type op basis van de extensie. Paden met lege delen, `.`, `..` of backslashes worden geweigerd. `data.json` is nooit een publiek asset.

Fouten: `404 game_not_found` (onbekend, uitgezet of geblokkeerd), `404 asset_not_found`

`GET /releases/{id}/{version}/assets/{path}`

Toegang openbaar

Een publiek bestand uit één bewaarde versie van een geüploade game, met dezelfde padregels. Alleen de actieve versie van een game die aan staat, of een versie die een lopende kamer nog speelt, wordt geserveerd. Zo laden de pagina's van een kamer hun assets uit hun eigen versie, ook als de ontwikkelaar de game uitzet.

Fouten: `404 game_not_found` (onbekend of geblokkeerd), `404 release_not_found`, `404 asset_not_found` (ook voor een ongebruikte, niet-actieve versie)

Pagina’s en SDK buiten de API

Pad	Inhoud
`/`	Tijdelijke doorverwijzing naar `/home`, de startpagina.
`/home`, `/home/join`, `/home/account`, `/home/new/{id}`	Pagina's om games te kiezen, kamers te maken, binnen te komen en je account te beheren. De start toont meedoen met een code in zes tegels en vier spelkaarten; bij meer dan vier spellen vier willekeurige, met een link naar `/home/games`. De accountpagina heeft "Mijn spellen" als het account eigen spellen heeft.
`/home/games`	Alle spellen als kaarten: het icoon op de eigen kleur van het spel, het aantal spelers, of het met een tv kan, eigen sets en het aantal achievements. De hele kaart opent `/home/new/{id}`. Tot 0.6.1 heette deze pagina `/home/spellen`.
`/home/join?room=CODE`	De meedoelink: opent het meedoen met de code al ingevuld. Ook `/home?room=CODE` stuurt hierheen.
`/home/sets`, `/home/sets/{id}`	Mijn sets: eigen sets bekijken, maken (`/home/sets/new?game=…&kind=…`), bewerken en verwijderen. Na het bewaren opent `/home/sets/{id}?saved` met een korte bevestiging.
`/demo/…`	Permanente doorverwijzing (308) naar hetzelfde pad onder `/home`, met de query erbij. Zo blijven oude links en bevestigingsmails werken.
`/room/{code}/{role}`	De kamerschil voor `host`, `player` of `display`, met het gameframe erin.
`/sdk/mini-engine.js`	De SDK-build die de server serveert, als ES-module.
`/sdk/mini-engine.d.ts`	De bijbehorende TypeScript-declaraties, als platte tekst.

Op de startpagina, `/home/games` en de pagina om een kamer te maken staat het icoon van een game als `<img alt="">`, zodat scripts in een SVG nooit draaien. De spelkaart neemt de kleur van de rand van de tegel over als vlak, zodat alleen de tekening te zien is. Zonder icoon tonen ze het eigen merk van een ingebouwde game, of een tegel met de eerste letter.

In de kamerschil is de kamercode in de balk een knop: die kopieert de meedoelink `{origin}/home/join?room=CODE` en toont anderhalve seconde "Gekopieerd". Ernaast opent "Toon QR" een venster met een grote QR-code van dezelfde link, de code en "Sluiten". Op het scherm staat in de lobby een QR-code naast de code, en de uitnodiging noemt alleen het adres van de site: "Doe mee op miniengine.nl met code …". Een eigen game kan dit overnemen met de MiniEngineUI-kit uit de SDK-skills (`kit/ui/invite.js`).

HTTP: kamers

Routes om kamers te maken, binnen te komen en het spel te bedienen. De meeste routes geven de bijgewerkte snapshot van jouw rol terug, zodat je meteen kunt tekenen.

Maken, deelnemen en lezen

`POST /api/v1/rooms`

Toegang gast, of een accounttoken om de stoel van de maker te koppelen

Maakt een kamer en geeft de hostcredentials terug. Alleen `game_id` is verplicht.

- `max_players`: standaard het maximum van de game, afgetopt op 24 en niet lager dan het minimum.
- `settings`: wordt samengevoegd met de standaardinstellingen van de game en daarna tegen het schema gecontroleerd.
- `config`: weggelaten velden worden de standaardmodus van de game, `display_enabled` volgens `has_display` en `allow_late_join:false`.
- `name`: naam van een meespelende maker (`playing` of `hostless`); standaard `Spelleider`.
- `set_id`: een eigen set van het ingelogde account voor deze game; vraagt een accounttoken. Zie *Eigen sets*.

De kamer onthoudt de actieve versie van de game als `game_version`, het account van de maker (als die was ingelogd) en een kopie van de gekozen set.

Verzoek

```
{
  "game_id": "common-dice",
  "name": "Dani",
  "max_players": 12,
  "settings": {"dice_count": 5, "sides": 6},
  "config": {"host_mode": "hostless", "display_enabled": true, "allow_late_join": false}
}
```

Antwoord 201

```
{
  "code": "TAFELS",
  "token": "HOST_SECRET",
  "role": "host",
  "player_id": "8c0e6c2a-…",
  "display_token": "DISPLAY_SECRET"
}
```

Een moderator krijgt `"player_id": null`. Bij `playing` en `hostless` is de credential van de maker zowel host als speler: `role` is `host` en `player_id` is zijn stoel. Dat geldt sinds 0.5.1 ook voor een browsergame. Bij een uitgeschakeld scherm ontbreekt `display_token`.

Fouten: `400 invalid_request` (spelersaantal, instellingen, modus of late joining die de game niet ondersteunt; `hostless` bij een browsergame; een set van een ander spel, of een soort of inhoud die niet bij de versie past), `400 invalid_json`, `401 unauthorized` (ongeldig accounttoken, of `set_id` zonder account), `404 game_not_found` (ook een uitgezette of geblokkeerde upload), `404 set_not_found`, `409 release_changed` (net een andere versie actief; probeer opnieuw), `429 rate_limit`, `503 room_limit`

`POST /api/v1/rooms/{code}/join`

Toegang gast, of een accounttoken om de stoel te koppelen

Komt de kamer binnen als speler. Namen zijn 1–24 tekens zonder controletekens en uniek binnen de kamer, ongeacht hoofdletters.

Verzoek

```
{"name": "Sander"}
```

Antwoord 201

```
{"code": "TAFELS", "token": "PLAYER_SECRET", "role": "player", "player_id": "3f2c9d71-…"}
```

Fouten: `400 invalid_request` (naam), `401 unauthorized`, `404 room_not_found`, `404 game_not_found` (game geblokkeerd), `409 room_started`, `409 room_full`, `409 name_taken`, `409 already_joined` (dit account zit al in de kamer; hervat de bestaande sessie), `429 rate_limit`

`GET /api/v1/rooms/{code}/info`

Toegang openbaar

Wat je moet weten voordat je binnenkomt. Geen namen, geen spelstate. `set` is `null`, of `{id, kind, title}` van de eigen set van de kamer.

Antwoord 200

```
{
  "code": "TAFELS",
  "game_id": "common-dice",
  "game_version": "2.0.0",
  "status": "lobby",
  "player_count": 3,
  "max_players": 12,
  "config": {"host_mode": "hostless", "display_enabled": true, "allow_late_join": false},
  "runtime": "server",
  "set": null
}
```

Fouten: `404 room_not_found`

`GET /api/v1/rooms/{code}`

Toegang elk kamertoken

De snapshot voor jouw rol. Dezelfde vorm komt over de websocket als `{"type":"snapshot","room":…}`. Hieronder de snapshot van een speler in Common Dice.

Antwoord 200


```
{
  "code": "TAFELS",
  "game_id": "common-dice",
  "game_version": "2.0.0",
  "status": "running",
  "version": 7,
  "max_players": 12,
  "settings": {
    "dice_count": 5, "sides": 6, "roll_permission": "everyone", "theme": "ivory"
  },
  "host_connected": true,
  "host_player_id": "8c0e6c2a-…",
  "players": [
    {"id": "8c0e6c2a-…", "name": "Dani", "connected": true, "ready": true,
     "account_id": null},
    {"id": "3f2c9d71-…", "name": "Sander", "connected": true, "ready": true,
     "account_id": "b71e0c55-…"}
  ],
  "public_state": {
    "results": [5, 3, 6, 2, 6], "roller": "Sander", "roller_id": "3f2c9d71-…",
    "sequence": 4, "configuration": "5d6", "held": [], "rolled_at": 1792656000000,
    "history": […]
  },
  "you": {"role": "player", "player_id": "3f2c9d71-…"},
  "runtime": "server",
  "config": {"host_mode": "hostless", "display_enabled": true, "allow_late_join": false},
  "round_id": "9b1d4f0e-…",
  "set": null,
  "private_state": null
}
```


De host krijgt daarnaast `host_state` en `player_states`; een display krijgt geen `private_state`. `host_connected` is alleen aanwezigheid: serveracties hangen er niet van af. `host_player_id` is de stoel van de maker als die meespeelt (`playing` of `hostless`), anders `null`; zo kan elke pagina de host aanwijzen. `game_version` is de versie waarmee de kamer is gemaakt; bij kamers van voor 0.3 is het `null`. `set` is `null`, of `{id, kind, title}` van de eigen set waarmee de kamer speelt; de inhoud staat nooit in een snapshot.

Fouten: `401 unauthorized`, `404 room_not_found`

`GET /api/v1/rooms/{code}/game-data`

Toegang hosttoken

De private speldata van de kamer, bijvoorbeeld vragen en antwoorden. Speelt de kamer met een eigen set, dan is dat de inhoud van die set, zoals de speler hem bewaarde. Anders is het `data.json` van de versie van de kamer. Alleen de host kan dit lezen.

Fouten: `401 unauthorized`, `403 forbidden`, `404 room_not_found`, `404 game_not_found` (game geblokkeerd)

Spelverloop

`POST /api/v1/rooms/{code}/ready`

Toegang deelnemer met een stoel, alleen in de lobby

Zet jouw readiness. Starten kan pas als iedere deelnemer klaarstaat. Verhoogt de versie niet. In een browsergame staat de stoel van een meespelende host al klaar; `ready` van die stoel mag nog, maar verandert alleen de vlag.

Verzoek

```
{"ready": true}
```

Antwoord 200

De bijgewerkte snapshot.

Fouten: `403 forbidden` (geen spelersstoel, zoals een moderator of display), `409 room_started`

`POST /api/v1/rooms/{code}/actions`

Toegang speler; bij een servergame ook de host als de regels dat toestaan

Stuurt een spelactie. De naam is 1–64 letters, cijfers, `_`, `.` of `-`; `data` is hoogstens 8 KiB en standaard `null`. Bij een servergame verwerkt de server de actie en slaat het resultaat op voordat het zichtbaar wordt. Bij een hostruntime moet de host verbonden zijn; de actie gaat dan naar zijn wachtrij. De host van een hostruntime stuurt zelf geen acties, ook niet als hij meespeelt: hij past zijn eigen zetten toe met `PUT …/state`.

Verzoek

```
{"name": "roll", "data": {"held": [0, 2]}, "action_id": "c0a8e1f4-…"}
```

Antwoord 200

De bijgewerkte snapshot.

Fouten: `400 invalid_request` (naam, grootte of `action_id`), `403 forbidden` (display, de host van een hostruntime, of een rol die deze actie niet mag), `409 game_not_running`, `409 host_offline`, `409 invalid_game_action`, `409 buzzer_closed`

`PUT /api/v1/rooms/{code}/state`

Toegang hosttoken, alleen bij `runtime:"host"`

Vervangt de volledige state. Weggelaten `host_state` wordt `null`, weggelaten `player_states` wordt `{}`. Sleutels in `player_states` moeten bestaande speler-ID's zijn. De state wordt opgeslagen voordat de nieuwe snapshot uitgaat. Servergames weigeren deze route, ook voor de host.

Verzoek

```
{
  "expected_version": 4,
  "public_state": {"round": 1},
  "host_state": {"answer": "Venus"},
  "player_states": {"3f2c9d71-…": {"cards": [1, 2]}}
}
```

Fouten: `400 invalid_request` (onbekende speler), `403 forbidden`, `409 version_conflict`, `413 state_too_large` (meer dan 64 KiB)

`PUT /api/v1/rooms/{code}/settings`

Toegang hosttoken

Vervangt het hele instellingenobject. De server controleert het schema van de game en daarna de spelregels; bij Common Dice wist een ander aantal of andere zijden de worpen, een ander thema niet.

Verzoek

```
{
  "expected_version": 7,
  "settings": {
    "dice_count": 3, "sides": 20, "roll_permission": "everyone", "theme": "obsidian"
  }
}
```

Fouten: `400 invalid_request`, `403 forbidden`, `409 version_conflict`, `409 teams_locked` (Jeopardy)

`POST /api/v1/rooms/{code}/lifecycle`

Toegang hosttoken

Zet de status door: `lobby → running → finished → lobby`. Naar `running` vereist dat alle deelnemers klaarstaan; de stoel van een meespelende host in een browsergame telt altijd als klaar. Jeopardy vraagt daarnaast een teamkeuze van iedere speler. Afronden van een servergame slaat ook de accountresultaten op.

Verzoek

```
{"status": "running"}
```

Fouten: `403 forbidden`, `409 invalid_transition`, `409 players_not_ready`, `409 teams_required`

Kamerbeheer

`PUT /api/v1/rooms/{code}/config`

Toegang hosttoken, alleen in de lobby

Past `display_enabled` en `allow_late_join` aan. De hostmodus ligt vast; onbekende velden worden afgewezen. Late joining kan alleen aan als de game `supports_late_join:true` heeft.

Verzoek

```
{"expected_version": 0, "config": {"display_enabled": false, "allow_late_join": true}}
```

Fouten: `400 invalid_request`, `403 forbidden`, `409 room_started`, `409 version_conflict`

`POST /api/v1/rooms/{code}/display-token`

Toegang hosttoken, scherm ingeschakeld

Maakt een nieuw displaytoken. Het vorige werkt meteen niet meer.

Antwoord 200

```
{"token": "DISPLAY_SECRET"}
```

Fouten: `403 forbidden` (geen host, of scherm uitgeschakeld)

`POST /api/v1/rooms/{code}/leave`

Toegang spelerstoken

Verlaat de kamer en trekt het eigen token in. Antwoord `204` zonder body. De host kan deze route niet gebruiken.

Fouten: `401 unauthorized`, `403 forbidden`

`DELETE /api/v1/rooms/{code}/players/{player_id}`

Toegang hosttoken

Verwijdert een speler, zijn private state en zijn token. Antwoord `204`. De maker kan zijn eigen stoel niet verwijderen.

Fouten: `400 invalid_request` (eigen stoel), `403 forbidden`, `404 player_not_found`

`DELETE /api/v1/rooms/{code}`

Toegang hosttoken

Verwijdert de kamer en sluit alle verbindingen met `room_deleted`. Antwoord `204`.

Fouten: `401 unauthorized`, `403 forbidden`, `404 room_not_found`

HTTP: accounts

Een account dat alle games op deze server delen, met een e-mailadres dat de speler via een link bevestigt. Geen externe inlog, geen wachtwoordherstel. Gastspel blijft altijd mogelijk. De routes voor achievements en sets van een account staan bij *Achievements* en *Eigen sets*.

Velden en regels

Veld	Regel
`username`	Wordt getrimd en naar kleine letters gezet; daarna 3–24 tekens uit a–z, 0–9 en `_`. Uniek.
`display_name`	Getrimd, 1–24 Unicode-tekens, geen controletekens.
`password`	Precies zoals ingevoerd, 10–128 UTF-8-bytes.
`email`	Verplicht bij registreren. Getrimd en naar kleine letters; een geldig adres van hoogstens 254 tekens. Een bevestigd adres hoort bij één account; een onbevestigd adres reserveert niets.

Het accountobject is `{id, username, display_name, created_at, email, email_verified}`. `email` is `null` bij een account zonder adres, zoals een account van vóór 0.4.

`POST /api/v1/accounts/register`

Toegang openbaar

Verzoek

```
{"username": "sander", "password": "dobbelsteen-42", "display_name": "Sander",
 "email": "sander@example.com"}
```

Antwoord 201

```
{
  "account": {
    "id": "b71e0c55-…", "username": "sander", "display_name": "Sander",
    "created_at": 1792656000, "email": "sander@example.com", "email_verified": false
  },
  "token": "ACCOUNT_SECRET"
}
```

`created_at` is in Unix-seconden. Het token wordt niet automatisch bewaard. De engine stuurt een bevestigingsmail; het account kan meteen inloggen en spelen. Staat de robotcheck aan, dan hoort het Turnstile-token in de header `X-Turnstile-Token`.

Fouten: `400 invalid_request`, `400 turnstile_required`, `400 turnstile_failed`, `409 username_taken`, `409 email_taken` (adres al bevestigd door een ander account), `429 rate_limit`, `503 turnstile_unavailable`

`POST /api/v1/accounts/login`

Toegang openbaar

Geeft dezelfde vorm als registreren, met status 200. Een onbekende naam en een fout wachtwoord geven dezelfde fout.

Verzoek

```
{"username": "sander", "password": "dobbelsteen-42"}
```

Fouten: `401 unauthorized`, `429 rate_limit`

`GET /api/v1/accounts/me`

Toegang accounttoken

Het accountobject: `{id, username, display_name, created_at, email, email_verified}`.

`POST /api/v1/accounts/logout`

Toegang accounttoken

Trekt deze sessie in. Antwoord `204`. Kamertokens die al zijn uitgegeven blijven geldig.

`GET /api/v1/accounts/stats`

Toegang accounttoken

Statistieken uit afgeronde serverrondes, eenmaal per account en ronde. Er is geen route om zelf scores te posten; eigen browsergames tellen niet mee.

Antwoord 200

```
{
  "totals": {"games_played": 12, "wins": 3},
  "games": [
    {"game_id": "jeopardy", "games_played": 5, "wins": 3,
     "total_score": 4200, "best_score": 1400, "counters": {"correct": 17, "wrong": 6}},
    {"game_id": "common-dice", "games_played": 7, "wins": 0,
     "total_score": 0, "best_score": 0,
     "counters": {"rolls": 41, "dice_rolled": 180, "total": 652}}
  ],
  "recent": [
    {"round_id": "9b1d4f0e-…", "game_id": "jeopardy", "room_code": "TAFELS",
     "player_id": "3f2c9d71-…", "account_id": "b71e0c55-…", "display_name": "Sander",
     "score": 1400, "won": true, "counters": {"correct": 5, "wrong": 1},
     "finished_at": 1792659600}
  ]
}
```

`recent` bevat de laatste twintig resultaten. Tellers tellen alleen gehele getallen op; scores kunnen negatief zijn. Common Dice kent geen winnaars toe.

Robotcheck

Vraagt de server om een robotcheck, dan willen `POST /api/v1/accounts/register` en `PUT /api/v1/accounts/email` een token van het Cloudflare Turnstile-widget, omdat ze een mail laten versturen. `GET /api/v1/accounts/config` geeft de sitekey om het widget mee te tonen. Stuur het token mee in de header `X-Turnstile-Token`, en geef het widget de actie `register` of `email`: de server accepteert een token alleen met de juiste actie en voor zijn eigen adres. Een token werkt één keer; vraag per poging een nieuw. Is de sitekey `null`, dan is de header niet nodig.

`GET` `/api/v1/accounts/config`

Toegang openbaar

De sitekey om het widget te tonen, of `null` als de robotcheck uit staat.

Antwoord 200

```
{"turnstile_site_key": "0x4AAAAAAA…"}
```

Status en code	Wanneer
`400 turnstile_required`	De check staat aan en de header ontbreekt of is leeg.
`400 turnstile_failed`	Cloudflare keurde het token af: verlopen, al gebruikt, een andere actie of een andere host.
`503 turnstile_unavailable`	Cloudflare was niet bereikbaar. Probeer het zo opnieuw.

E-mailbevestiging

De bevestigingsmail bevat een link naar `/home/account?verify=<token>` op het adres van de server en noemt het account (`@gebruikersnaam`). De accountpagina laat de speler eerst inloggen en stuurt het token daarna naar `verify-email`. Een oude link naar `/demo/account?verify=…` verwijst door en werkt nog.

- Een link werkt alleen voor het ingelogde account waarvoor hij is verstuurd. Wie zich met andermans adres registreert, krijgt het niet bevestigd doordat de eigenaar van het adres op de link klikt.
- Linktokens zijn willekeurig (256 bits). Een link verloopt na 48 uur, werkt één keer en vervalt zodra het adres verandert.
- Een onbevestigd adres reserveert niets. Het account dat als eerste bevestigt, houdt het; bij andere accounts met hetzelfde onbevestigde adres wordt `email` `null`.
- Hoogstens één bevestigingsmail per account per minuut.
- Zonder bevestigd adres kun je inloggen en spelen; alleen een dev-key vraagt een bevestigd adres.

`POST /api/v1/accounts/verify-email`

Toegang accounttoken van het account waarvoor de link is

Verzoek

```
{"token": "TOKEN_UIT_DE_LINK"}
```

Antwoord 200: `{"account": Account}` met `email_verified: true`.

Fouten: `400 invalid_token` (onbekend, verlopen, gebruikt, voor een ander adres of account), `401 unauthorized`, `409 email_taken`, `429 rate_limit`

`PUT /api/v1/accounts/email`

Toegang accounttoken

Voegt een adres toe, wijzigt het (weer onbevestigd) of stuurt de link opnieuw als het adres gelijk blijft. Er gaat altijd een nieuwe mail uit. Staat de robotcheck aan, dan hoort het token in `X-Turnstile-Token`.

Verzoek

```
{"email": "sander@example.com"}
```

Antwoord 200

```
{
  "account": {
    "id": "b71e0c55-…", "username": "sander", "display_name": "Sander",
    "created_at": 1792656000, "email": "sander@example.com", "email_verified": false
  },
  "verification_sent": true
}
```

Fouten: `400 invalid_request`, `400 turnstile_required`, `400 turnstile_failed`, `401 unauthorized`, `409 email_already_verified`, `409 email_taken`, `429 rate_limit` (al een mail in de afgelopen minuut, of de mailgrens per IP-adres), `503 turnstile_unavailable`

Alle accountroutes met een token geven `401 unauthorized` bij een ontbrekend, ongeldig of verlopen token.

Ontwikkelaars

Routes onder `/api/v1/dev` waarmee een ontwikkelaar met een dev-key zijn eigen browserhost-games uploadt, versies beheert en games aan- of uitzet. De `me`-CLI gebruikt precies deze routes. Aan- en uitzetten kan ook met een accounttoken, onder `/api/v1/accounts/games`.

Stuur de dev-key mee als `Authorization: Bearer med_…`. Een ongeldige, verlopen of ingetrokken key geeft `401 unauthorized`. Je ziet en beheert alleen je eigen games; een game van iemand anders geeft `404 game_not_found`.

Objecten

Een **DevGame** beschrijft één geüploade game met al zijn bewaarde versies, nieuwste eerst. `title` komt uit het manifest van de actieve versie.

```
{
  "id": "quiz-night",
  "title": "Quiz Night",
  "enabled": true,
  "blocked": false,
  "active_version": "1.1.0",
  "created_at": 1792660000,
  "updated_at": 1792663600,
  "versions": [Release, …]
}
```

Een **Release** is één bewaarde versie. `size` is in bytes, tijden zijn Unix-seconden. `active` is de versie die nieuwe kamers krijgen; `in_use` is `true` zolang een lopende kamer die versie speelt.

```
{
  "version": "1.1.0",
  "title": "Quiz Night",
  "content_hash": "9f2c…",
  "size": 48213,
  "file_count": 7,
  "created_at": 1792663600,
  "active": true,
  "in_use": false
}
```

Account en games

`GET /api/v1/dev/me`

Toegang dev-key

Het account achter de key en de key zelf, zonder geheim.

Antwoord 200

```
{
  "account": {"id": "b71e0c55-…", "username": "sander", "display_name": "Sander"},
  "token": {"id": "4d7a2c19-…", "label": "laptop", "expires_at": null}
}
```

`GET /api/v1/dev/games`

Toegang dev-key

Jouw games: `{"games": [DevGame, …]}`, gesorteerd op id.

`GET /api/v1/dev/games/{id}`

Toegang dev-key

Eén DevGame.

Fouten: `404 game_not_found` (onbekend, of niet van jou)

`PUT /api/v1/dev/games/{id}`

Toegang dev-key

Zet de game uit of aan en geeft de DevGame terug. Een uitgezette game verdwijnt uit `GET /api/v1/games` en `/home`, er kunnen geen nieuwe kamers mee worden gemaakt en de HTML en assets zonder kamer geven 404. Lopende kamers spelen door via hun eigen versie.

Verzoek

```
{"enabled": false}
```

Fouten: `403 game_blocked` (aanzetten terwijl de beheerder de game blokkeerde), `404 game_not_found`

`DELETE /api/v1/dev/games/{id}`

Toegang dev-key

Verwijdert de game met alle versies. Antwoord `204`. Het id is daarna weer vrij.

Fouten: `404 game_not_found`, `409 game_in_use` (een kamer speelt de game nog), `403 game_blocked` (een geblokkeerde game kan alleen de beheerder vrijgeven)

Mijn spellen op de accountpagina

Sinds 0.5.1 zet een ingelogde ontwikkelaar zijn eigen spellen ook op `/home/account` aan of uit, onder "Mijn spellen". De pagina gebruikt daarvoor de accountsessie (`Authorization: Bearer ACCOUNT_SECRET`, net als `/api/v1/accounts/me`), geen dev-key. Per spel toont ze de titel, de actieve versie, de toestand ("Aan", "Uit" of "Geblokkeerd door de beheerder") en een knop "Zet uit" of "Zet aan", behalve bij een geblokkeerd spel.

`GET` `/api/v1/accounts/games`

Toegang accounttoken

De geüploade games van dit account: `{"games": [DevGame, …]}`, in dezelfde vorm als `GET /api/v1/dev/games`. Een account zonder eigen games krijgt een lege lijst.

Fouten: `401 unauthorized`

`PUT` `/api/v1/accounts/games/{id}`

Toegang accounttoken

Zet een eigen game aan of uit en geeft de DevGame terug, met hetzelfde gevolg als `PUT /api/v1/dev/games/{id}`.

Verzoek

```
{"enabled": false}
```

Fouten: `401 unauthorized`, `403 game_blocked` (aanzetten terwijl de beheerder de game blokkeerde), `404 game_not_found` (onbekend, of niet van jou)

Een versie uploaden

`POST` `/api/v1/dev/games/{id}/versions`

Toegang dev-key; body hoogstens 16 MiB

Uploadt een nieuwe versie: alle bestanden van het pakket als base64, met het pad als sleutel. De versie is het `version` uit `manifest.json`. Bestaat de game nog niet, dan wordt hij van jou en is deze eerste versie meteen actief. Latere versies worden alleen actief met `"activate": true`.

Verzoek

```
{
  "files": {
    "manifest.json": "eyJpZCI6InF1aXotbmlnaHQiLC4uLn0=",
    "host.html": "PCFkb2N0eXBlIGh0bWw+…",
    "player.html": "PCFkb2N0eXBlIGh0bWw+…",
    "display.html": "PCFkb2N0eXBlIGh0bWw+…",
    "assets/app.js": "aW1wb3J0IHsgTWluaUVuZ2luZSB9…"
  },
  "activate": true
}
```

Antwoord 201

```
{
  "game": {"id": "quiz-night", "title": "Quiz Night", "enabled": true, "blocked": false,
           "active_version": "1.1.0", "created_at": 1792660000, "updated_at": 1792663600,
           "versions": [{"version": "1.1.0", …}, {"version": "1.0.0", …}]},
  "release": {"version": "1.1.0", "title": "Quiz Night", "content_hash": "9f2c…",
              "size": 48213, "file_count": 7, "created_at": 1792663600,
              "active": true, "in_use": false}
}
```

Regels voor een upload

Een upload volgt dezelfde regels als een ingebouwde game: manifest, rolpagina's, instellingenschema, hoogstens 4096 bestanden, 2 MiB per bestand en 10 MiB samen. Daarbij komt:

- Het `id` in het manifest is gelijk aan het id in de URL.
- `version` is 1–64 tekens uit `0-9 A-Z a-z . + _ -`, begint met een letter of cijfer en is nieuw voor deze game. Een versie overschrijven kan niet.
- Alleen `runtime:"host"`, met `host_modes` uit `playing`, `tv` en `moderator`; `hostless` kan niet.
- Een `icon` volgt de regels bij *Manifest en instellingen*; anders volgt `Invalid icon: …`.
- `achievements` en `set_kinds` volgen dezelfde regels als bij een ingebouwde game, inclusief het set-schema. Zie *Achievements* en *Eigen sets*.
- Paden zijn relatief, met `/`, hoogstens 255 tekens en 32 delen, zonder lege delen, `.`, `..`, backslashes of controletekens.
- Hoogstens 10 games per account en 10 bewaarde versies per game.
- Een mislukte upload laat niets achter. Elke versie krijgt een contenthash over paden en inhoud, die je terugziet als `content_hash`.

Fouten bij uploaden

Status en code	Wanneer
`400 invalid_request`	Ongeldig pakket: manifest, rolpagina's, instellingenschema, achievements, soorten sets, icoon, pad, base64, versienaam, een `runtime` anders dan `host`, `hostless` als hostmodus, of te groot.
`400 invalid_json`	Geen geldige JSON of onbekende velden in de body.
`403 forbidden`	Het id hoort bij een game van een andere ontwikkelaar.
`409 game_id_taken`	Het id hoort bij een ingebouwde game.
`409 version_exists`	Deze versie bestaat al voor deze game. Verhoog `version` in `manifest.json`.
`409 quota_games`	Je hebt al 10 games. Verwijder er eerst een.
`409 quota_releases`	Deze game heeft al 10 versies. Verwijder eerst oude versies.
`413 invalid_json`	De body is groter dan 16 MiB.

Versies beheren

`POST /api/v1/dev/games/{id}/versions/{version}/activate`

Toegang dev-key

Maakt een bewaarde versie actief, ook om terug te gaan naar een oudere. De engine controleert de versie eerst opnieuw, zodat een beschadigde versie nooit actief wordt. Geeft de DevGame terug. Lopende kamers houden hun eigen versie.

Fouten: `404 game_not_found`, `404 release_not_found`

`DELETE /api/v1/dev/games/{id}/versions/{version}`

Toegang dev-key

Verwijdert één bewaarde versie. Antwoord `204`.

Fouten: `404 game_not_found`, `404 release_not_found`, `409 release_active` (activeer eerst een andere versie), `409 release_in_use` (een lopende kamer speelt deze versie nog)

Lokaal testen met me dev

Sinds CLI 0.5.0 test je een game met `runtime:"host"` op je eigen computer. `me dev` controleert `manifest.json` zoals `me game upload` en start een kleine server die Mini Engine nabootst, met dezelfde routes als in deze referentie: kamers, spelers en klaar zijn, state met versies, acties, realtime, achievements en game-data.

```
me dev [map] [--port 3170] [--players 2] [--set bestand.json] [--no-open]
```

Eén overzicht in de browser toont de spelleider, het scherm en de spelers naast elkaar; elke stoel heeft een eigen poort en dus een eigen sessie. Een wijziging in de map laadt de stoelen opnieuw en de kamer blijft bestaan. Alles staat in het geheugen en de server luistert alleen op `127.0.0.1`. `me dev` is een

nabootsing: test een game voor een avond met vrienden ook een keer als testversie op de server. De eerste upload van een nieuwe game is meteen actief; latere versies worden pas actief met `--activate`.

Versies en kamers

Elke upload is een vaste versie. Een kamer speelt de versie die actief was toen hij werd gemaakt, ook als de ontwikkelaar daarna een nieuwe activeert, iemand de pagina ververst of later aansluit.

De versie van een kamer

Bij het maken van een kamer bewaart de engine de actieve versie als `game_version`. Het veld staat in `GET /api/v1/rooms/{code}/info` en in elke snapshot. Instellingen controleren, `game-data`, meedoen en verbinden gebruiken die versie. Bij kamers van voor 0.3 is `game_version` `null`.

Ingebouwde games krijgen het `version` uit hun manifest, maar worden bij een nieuwe versie van Mini Engine op hun plaats vervangen. Voor hen is `game_version` informatief.

Ook achievements komen uit de versie van de kamer: de server en de host kennen alleen achievements toe die die versie declareert. Sets gaan tegen de actieve versie: bewaren en wijzigen controleren de soort en het schema van de actieve versie, en een kamer met een eigen set krijgt de versie waartegen de set net is gecontroleerd. Werd intussen een andere versie actief, dan volgt `409 release_changed`.

Vaste HTML en assets

De kamerschil laadt de rolpagina met `?room=CODE`. De engine zoekt de versie van die kamer op en serveert de HTML van die versie. Bij een geüploade game vervangt hij in die HTML elk voorkomen van `/games/{id}/assets/` door het pad van de vaste versie:

```
GET /api/v1/games/quiz-night/player?room=TAFELS
/games/quiz-night/assets/app.js  →  /releases/quiz-night/1.0.0/assets/app.js
```

Hoort de kamer bij een andere game, of bestaat hij niet, dan negeert de engine `?room=` en serveert hij de actieve versie.

Relatieve imports

De engine past alleen de HTML aan, niet je JavaScript. Schrijf imports tussen je eigen modules daarom relatief, zodat ze uit dezelfde versie komen:

```
import { score } from './logic.js';                          // goed: dezelfde versie
import { score } from '/games/quiz-night/assets/logic.js';  // fout: altijd de actieve versie
```

Aan, uit en geblokkeerd

De ontwikkelaar zet een game aan of uit met de `me`-CLI, `PUT /api/v1/dev/games/{id}` of, sinds 0.5.1, onder "Mijn spellen" op de accountpagina.

Toestand	Catalogus en nieuwe kamers	Zonder kamer	Lopende kamers
Aan	In `GET /api/v1/games`, `/home` en `/home/games`	HTML en assets van de actieve versie	Spelen hun eigen versie
Uitgezet door de ontwikkelaar	Weg; nieuwe kamers geven `404 game_not_found`	HTML zonder `?room=` en `/games/{id}/assets/…` geven `404 game_not_found`	Spelen door via `?room=` en `/releases/{id}/{version}/…`
Geblokkeerd door de beheerder	Weg; nieuwe kamers geven `404 game_not_found`	`404 game_not_found`	Stoppen: HTML, assets, releaseassets en `game-data` geven 404, meedoen geeft `404 game_not_found`

Opruimen

Een game bewaart hoogstens 10 versies. Verwijderen kan niet bij de actieve versie (`409 release_active`) en niet bij een versie die een lopende kamer nog speelt (`409 release_in_use`). Een hele game verwijderen kan pas als geen enkele kamer hem meer speelt (`409 game_in_use`). Een kamer telt mee tot hij verloopt, twaalf uur na zijn laatste activiteit.

Websocket

Eén verbinding per kamertoken. De server stuurt na elke wijziging een verse snapshot voor jouw rol; jij stuurt opdrachten en krijgt per opdracht een bevestiging of een fout. Over dezelfde verbinding lopen de realtimeberichten.

Verbinden

```
GET /api/v1/rooms/{code}/ws   (upgrade naar websocket)
```

Stuur binnen vijf seconden als eerste tekstframe het token. Daarna volgt de eerste snapshot, en direct daarna één keer `rt.values` met de huidige realtimewaarden van de kamer.

```
→ {"type": "auth", "token": "PLAYER_SECRET"}
← {"type": "snapshot", "room": {…}}
← {"type": "rt.values", "values": […]}
```

Alleen tekstframes met JSON; binaire frames geven een fout. Na vijf seconden zonder auth volgt `auth_timeout` en sluit de server. Bestaat de kamer of de game niet (meer), of is de game geblokkeerd, dan faalt de upgrade met een gewone HTTP-fout.

Van client naar server

Elk bericht heeft een `type` en mag een `request_id` van hoogstens 64 tekens meesturen. De server zet die terug in de `ack` of `error`.

type	Velden	Gelijk aan
`action`	`name`, `data`, optioneel `action_id`	`POST …/actions`
`ready`	`ready`	`POST …/ready`
`configure`	`expected_version`, gedeeltelijke `config`	`PUT …/config`
`set_state`	`expected_version`, `public_state`, optioneel `host_state`, `player_states`	`PUT …/state`
`settings`	`expected_version`, volledig `settings`	`PUT …/settings`
`lifecycle`	`status`	`POST …/lifecycle`
`award`	`player_id`, `achievement_id`	`POST …/achievements`; alleen een `ack`
`ping`	geen	alleen een `ack`
`rt.emit`	`channel`, optioneel `data`, `to`, `echo`	realtime-event; geen `ack`
`rt.set`	`key`, `value` (`null` wist)	realtimewaarde; geen `ack`
`rt.reset`	geen; alleen de host	wist alle waarden; geen `ack`

Realtimeberichten (`rt.*`) kennen geen `request_id` en krijgen nooit een `ack`. Gaat er iets mis, dan volgt een `error` zonder `request_id`. Ze tellen niet mee in het gewone berichtenbudget; zie *Realtime*.

Van server naar client

type	Wanneer	Vorm
`snapshot`	Na auth, na elke wijziging in de kamer en na een eigen mutatie.	`{"type":"snapshot","room":{…}}`
`ack`	Een opdracht is gelukt.	`{"type":"ack","request_id":"1"}`
`error`	Een opdracht is mislukt, of er ging iets mis buiten een opdracht.	`{"type":"error","request_id":"1","error":{"code":"…","message":"…"}}`
`action`	Alleen voor de host van een hostruntime: een speler stuurde een actie.	`{"type":"action","player_id":"…","name":"vote","data":{…},"request_id":"3"}`
`closed`	De server sluit de verbinding.	`{"type":"closed","reason":"room_deleted"}`
`rt.values`	Eén keer, direct na de eerste snapshot van een verbinding: alle huidige waarden.	`{"type":"rt.values","values":[{"key":"kart","value":{…},"from":{…},"ts":…}]}`
`rt.event`	Iemand stuurde een event dat jou bereikt.	`{"type":"rt.event","channel":"buzz","data":{…},"from":{"role":"player","player_id":"…"},"ts":1792656000123}`
`rt.value`	Een ander zette een waarde, of wiste hem (`value: null`).	`{"type":"rt.value","key":"kart","value":{…},"from":{…},"ts":…}`
`rt.reset`	De host wiste alle waarden.	`{"type":"rt.reset","from":{"role":"host","player_id":null},"ts":…}`
`rt.throttled`	De server gooide realtimeberichten van jou weg; hoogstens eens per vijf seconden.	`{"type":"rt.throttled","dropped":12}`
`achievement`	Een speler in de kamer verdiende een achievement voor het eerst.	`{"type":"achievement","player_id":"…","achievement":{"id":"winnaar","title":"Winnaar","description":"…"}}`

Na een mutatie komt eerst de bijgewerkte snapshot en dan de `ack` ; de ack bevat zelf geen state. Bij een hostruntime bevestigt de ack van een speleractie alleen dat de actie in de wachtrij van de host staat, niet dat het spel haar heeft verwerkt. Realtimeberichten en `achievement` hebben geen vaste volgorde ten opzichte van snapshots en staan nooit in een snapshot.

Een ronde in berichten

```
→ {"type": "ready", "ready": true, "request_id": "1"}
← {"type": "snapshot", "room": {"status": "lobby", "version": 3, …}}
← {"type": "ack", "request_id": "1"}
→ {"type": "action", "name": "roll", "data": {"held": []}, "action_id": "c0a8e1f4-…",
"request_id": "2"}
← {"type": "snapshot", "room": {"status": "running", "version": 5, …}}
← {"type": "ack", "request_id": "2"}
```

Sluitredenen

reason	Betekenis	Wat de SDK doet
`session_replaced`	Hetzelfde token verbond opnieuw, bijvoorbeeld in een ander tabblad.	Stopt; credentials blijven bewaard voor expliciet hervatten.
`session_revoked`	Het token is ingetrokken: vertrokken, verwijderd of het scherm is uitgezet.	Stopt.
`room_deleted`	De host heeft de kamer verwijderd.	Stopt.
`room_expired`	Twaalf uur geen activiteit.	Stopt.
`host_lagged`	De host liep te ver achter op de gebeurtenissen.	Verbindt opnieuw met backoff.
`heartbeat_timeout`	45 seconden geen verkeer van de client.	Verbindt opnieuw met backoff.
`rate_limit`	Te veel gewone berichten per seconde. Realtime sluit nooit de verbinding.	Verbindt opnieuw na een extra pauze.
`server_shutdown`	De server stopt of herstart.	Verbindt opnieuw met backoff.

De server stuurt elke vijftien seconden een websocket-ping. Spelers en schermen die achterlopen krijgen gewoon een verse snapshot. Onzekere opdrachten worden nooit automatisch herhaald.

Realtime

Naast de opgeslagen state heeft elke kamer een snel kanaal voor dingen die over een seconde niet meer tellen: een buzzer, een emoji, de positie van een kart. Realtimeberichten gaan over de gewone kamerwebsocket, worden nooit bewaard en verhogen de versie niet.

Events en waarden

Soort	Gedrag	Voor
Events (`rt.emit`)	Eenmalig, eventueel naar een deel van de kamer. Geen bevestiging.	Buzzers, reacties, geluiden, een animatie die klaar is.
Waarden (`rt.set`)	De laatste waarde wint, per afzender en sleutel. Wie later verbindt, krijgt de huidige waarden.	Posities, cursors, wie aan het typen is.

Iedere rol mag events sturen en waarden zetten: de host, de spelers en het scherm. Realtime werkt in elke kamerstatus. Er is geen vaste volgorde ten opzichte van snapshots, en als het druk is kan een bericht wegvallen. Dat hoort bij het contract: laat de uitslag nooit van realtime afhangen. Bij Jeopardy blijft `buzz` de actie die telt; het event `pressed` laat alleen sneller zien wie drukte.

Van client naar server

```
{"type": "rt.emit", "channel": "buzz", "data": {"clue": "c2-300"}, "to": "all"}
{"type": "rt.set", "key": "kart", "value": {"x": 120, "y": 48}}
{"type": "rt.set", "key": "kart", "value": null}
{"type": "rt.reset"}
```

- `channel` en `key`: 1–64 tekens uit letters, cijfers en `_ . : -`.
- `data` en `value`: elke JSON-waarde van hoogstens 2 KiB, zoals de server hem serialiseert. `data` is standaard `null`; `value: null` wist de sleutel.
- `to` (alleen bij events): `"all"` (standaard), `"host"`, `"players"`, `"display"` of een lijst van hoogstens 16 speler-ID's.
- De afzender krijgt zijn eigen event niet terug, tenzij hij `"echo": true` meestuurt.
- `rt.reset` mag alleen de host. Het wist alle waarden in de kamer.
- Er is geen `request_id` en geen `ack`. Een onbekend veld, ook `from`, `ts` of `request_id`, een ongeldige naam, te veel data of een onbekende `to` geeft een `error` met `invalid_request`. `rt.reset` van een andere rol geeft `forbidden`.

Van server naar client

```
{"type": "rt.event", "channel": "buzz", "data": {"clue": "c2-300"},
 "from": {"role": "player", "player_id": "3f2c9d71-…"}, "ts": 1792656000123}
{"type": "rt.value", "key": "kart", "value": {"x": 120, "y": 48},
 "from": {"role": "player", "player_id": "3f2c9d71-…"}, "ts": 1792656000140}
{"type": "rt.values", "values": [{"key": "kart", "value": {"x": 120, "y": 48},
 "from": {"role": "player", "player_id": "3f2c9d71-…"}, "ts": 1792656000140}]}
{"type": "rt.reset", "from": {"role": "host", "player_id": null}, "ts": 1792656005000}
{"type": "rt.throttled", "dropped": 12}
```

- `from` en `ts` zet de server. `from` is de rol en de speler-ID van de afzender; een meespelende host stuurt als `{"role": "host", "player_id": "…"}`. `ts` is het moment waarop de engine het bericht ontving, in milliseconden sinds 1 januari 1970 (UTC). Zo vergelijk je alle telefoons op één klok, maar netwerkvertraging telt mee: het is de aankomsttijd, niet het moment van drukken. Een client kan geen van beide opgeven.
- `rt.values` komt één keer, direct na de eerste snapshot van een verbinding, met alle waarden van dat moment. Daarna houden `rt.value`-berichten je bij.
- Je eigen `rt.value` krijg je niet terug. Na opnieuw verbinden staan je eigen waarden wel in `rt.values`.
- `rt.reset` gaat naar iedereen, ook naar de host die het stuurde.

Ontvangers

`to`	Bereikt
`"all"`	Iedere verbinding in de kamer.
`"host"`	Alleen de host.
`"players"`	Iedereen met een spelersstoel, ook een meespelende host.
`"display"`	Alleen het gedeelde scherm.
`["3f2c9d71-…", "8c0e6c2a-…"]`	De spelers met deze ID's, hoogstens 16.

De afzender valt er steeds af, tenzij `echo` aan staat. Waarden gaan altijd naar de hele kamer.

Waarden bewaren

De server houdt waarden bij per afzender en sleutel. Afzenders zijn `host`, `display` of een speler-ID; niemand kan de waarden van een ander overschrijven. Waarden staan alleen in het geheugen. Ze verdwijnen als een speler vertrekt of wordt verwijderd (de kamer krijgt dan per sleutel een `rt.value` met `value: null`), als de host `rt.reset` stuurt, en als de kamer sluit of verloopt of de server herstart. Wie even de verbinding verliest, houdt zijn waarden.

Grenzen

- Per verbinding mag je 60 realtimeberichten per seconde sturen, met een buffer van 120. Dat budget staat los van dat voor gewone berichten.
- Daarboven gooit de server berichten zonder fout weg en meldt hij het aantal met `rt.throttled`, hoogstens eens per vijf seconden. De verbinding sluit nooit door realtime.

- Hoogstens 64 sleutels per afzender en 256 KiB aan waarden per kamer. Een `rt.set` daarboven valt weg en telt mee in `rt.throttled`.
- Een ontvanger die achterloopt, mist realtimeberichten. Zijn snapshots lopen gewoon door: geen `host_lagged` en geen extra snapshot.
- Een vervangen of ingetrokken verbinding kan meteen niets meer sturen.

Een buzzer in de SDK

```
// Speler: laat meteen zien dat je drukte; de actie telt
client.realtime.emit('pressed', { clue: 'c2-300' });
await client.action('buzz');

// Host en scherm: wie drukte, op volgorde van aankomst, met het verschil in ms
client.realtime.on('pressed', (data, from, ts) => {
  presses.push({ player: from.player_id, ts });
});

// Live waarde: per sleutel samengevoegd, hoogstens eens per 33 ms verstuurd
client.realtime.set('kart', { x: 120, y: 48 });
client.realtime.onValue('kart', (value, from) => drawKart(from.player_id, value));
```

Alle leden van `client.realtime` staan bij *SDK-client*.

Achievements

Een game declareert achievements in zijn manifest. Spelers met een account verdienen ze en houden ze op hun account. Gasten, een spelleider zonder stoel en het scherm verdienen niets.

Declareren

```
"achievements": [
  {"id": "winnaar", "title": "Winnaar", "description": "Win een show met je team."}
]
```

- Optioneel, hoogstens 50 per game.
- `id`: 1–48 tekens uit kleine letters, cijfers, `_` en `-`, beginnend met een letter of cijfer (`^[a-z0-9][a-z0-9_-]{0,47}$`), uniek binnen de game.
- `title` 1–60 tekens en `description` 1–160 tekens, zonder controletekens.
- De engine controleert dit voor ingebouwde games, bij een upload en bij het laden van een bewaarde versie. De lijst is openbaar via `GET /api/v1/games`.

Wie ze toekent

Runtime	Wie beslist	`source`
`server`: Common Dice en Jeopardy	De server, als de ronde van `running` naar `finished` gaat, samen met de resultaten.	`server`
`host`: eigen games	De host, met het websocketbericht `award` of `POST /api/v1/rooms/{code}/achievements`.	`game`

Het account komt altijd uit de eigen spelersgegevens van de server, nooit uit een client. Een achievement verdien je één keer per account en game: nog eens toekennen verandert niets en slaagt toch. De engine bewaart titel en beschrijving op het moment van toekennen, zodat ze blijven staan als de game later verandert of verdwijnt. Hij kent alleen achievements toe die de versie van de kamer declareert.

Ingebouwde achievements

Game	`id`	Wanneer
Jeopardy	`eerste-antwoord`	Minstens één goed antwoord in de show.
Jeopardy	`winnaar`	Je team heeft de hoogste score en scoorde punten. Bij gelijke stand winnen alle teams met die score.
Jeopardy	`foutloos`	Minstens drie goede antwoorden en geen fout in één show.
Jeopardy	`eigen-bord`	Je speelde in een team een show op een eigen vragenbord.
Common Dice	`dubbel-zes`	Minstens twee zessen in één worp.
Common Dice	`hoogste-worp`	De hoogste worp van de ronde; bij gelijke stand iedereen met die worp.

Toekennen vanuit een eigen game

Bij een browserhost-game draait de spelcode in de browser van de host. Die kan een achievement van de eigen game toekennen aan een speler in de kamer, over de websocket:

```
→ {"type": "award", "player_id": "3f2c9d71-…", "achievement_id": "snelste",
   "request_id": "7"}
← {"type": "ack", "request_id": "7"}
```

De server controleert:

- de afzender is de host, de game heeft `runtime:"host"` en de kamer is `running` of `finished`;
- de speler zit in de kamer en speelt met een account;
- de versie van de kamer declareert het achievement;
- de maker was ingelogd toen hij de kamer maakte, en de ontvanger is niet dat account, behalve op de eigen stoel van een meespelende host (`host_player_id`);
- hoogstens 30 toekenningen per kamer per minuut, ook als ze al verdiend waren.

Een meespelende host is een speler als ieder ander: zijn eigen stoel kan een achievement krijgen. Een spelleider die met hetzelfde account apart als speler meedoet, kan zichzelf niets geven. In een `tv`-kamer kan dat wel: de tv is een apparaat, dus wie op de tv is ingelogd, krijgt op de eigen telefoon gewoon achievements.

`POST /api/v1/rooms/{code}/achievements`

Toegang hosttoken, alleen bij `runtime:"host"`

Hetzelfde als `award`, over HTTP. `already_earned` is `true` als de speler het achievement al had.

Verzoek

```
{"player_id": "3f2c9d71-…", "achievement_id": "snelste"}
```

Antwoord 200

```
{
  "player_id": "3f2c9d71-…",
  "achievement": {"id": "snelste", "title": "Snelste",
                  "description": "Geef het snelste goede antwoord."},
  "already_earned": false
}
```

Fouten: `401 unauthorized`, `403 forbidden` (geen host, of een servergame), `403 not_eligible` (gast, kamer gemaakt zonder account, of het account van de maker op een andere stoel dan die van de meespelende host), `404 room_not_found`, `404 game_not_found` (game geblokkeerd), `404 player_not_found`, `404 achievement_unknown` (niet gedeclareerd in de versie van de kamer), `409 game_not_running` (kamer in de lobby), `429 rate_limit`

Vertrouwen. Achievements uit een eigen game controleert de server niet: de spelcode draait bij de host, en een spelleider kan de achievements van die game toekennen aan anderen in zijn kamer. Daarom bewaart de engine ze met `source: "game"` en toont de accountpagina ze apart, onder 'Toegekend door eigen spellen'. Resultaten en statistieken komen alleen uit servergames.

Vieren in de kamer

Verdient een speler een achievement voor het eerst, dan gaat dat meteen naar de hele kamer. Het bericht staat niet in de snapshot en verhoogt de versie niet. Een achievement dat al verdiend was, geeft geen bericht.

```
{"type": "achievement", "player_id": "3f2c9d71-…",
 "achievement": {"id": "winnaar", "title": "Winnaar",
                 "description": "Win een show met je team."}}
```

In de SDK: `client.on('achievement', ({ player_id, achievement }) => …)`.

`GET` `/api/v1/accounts/achievements`

Toegang accounttoken

Alle achievements van het account, nieuwste eerst. `game_title` is de huidige titel van de game, of `null` als de game niet meer bestaat of geblokkeerd is. `awarded_at` is in Unix-seconden.

Antwoord 200

```
{
  "achievements": [
    {"game_id": "jeopardy", "game_title": "Jeopardy", "id": "winnaar",
     "title": "Winnaar", "description": "Win een show met je team.",
     "source": "server", "awarded_at": 1792659600},
    {"game_id": "quiz-night", "game_title": "Quiz Night", "id": "snelste",
     "title": "Snelste", "description": "Geef het snelste goede antwoord.",
     "source": "game", "awarded_at": 1792658000}
  ]
}
```

Fouten: `401 unauthorized`

De accountpagina (`/home/account`) toont de achievements van de server per game, en alleen als die er zijn de groep 'Toegekend door eigen spellen'.

Eigen sets

Een game kan spelers eigen inhoud laten maken, zoals een eigen vragenbord. De game declareert welke soorten sets er zijn, elk met een schema. Spelers bewaren sets op hun account en kiezen er een als ze een kamer maken.

Soorten sets in het manifest

```
"set_kinds": [
  {"id": "board", "title": "Vragenbord",
   "description": "Eén tot zes categorieën met elk vijf vragen.",
   "schema": {"type": "object", "properties": {…}, "required": ["categories"],
              "additionalProperties": false}}
]
```

- Optioneel, hoogstens 4 soorten per game.
- `id` volgt dezelfde regel als een achievement-id; `title` 1–60 en `description` 1–160 tekens.
- `schema` gebruikt het set-schema hieronder. Het is openbaar via `GET /api/v1/games`, zodat een editor er velden uit kan maken.

Het set-schema

Een begrensd deel van JSON Schema met een eigen, eenvoudige controle. Het staat los van het instellingenschema, dat plat blijft. Elk onderdeel heeft een `type` en daarbij alleen deze sleutels:

`type`	Sleutels
`object`	`properties`, optioneel `required`, en verplicht `additionalProperties: false`
`array`	`items` (verplicht), `minItems`, `maxItems`
`string`	`minLength`, `maxLength` (in tekens), `enum` (unieke teksten)
`integer`, `number`	`minimum`, `maximum`
`boolean`	geen

- Elk onderdeel mag daarnaast `title` en `description` hebben, als tekst. Andere sleutels, zoals `default`, `pattern` of `$ref`, worden geweigerd.
- Hoogstens 8 niveaus diep en 256 onderdelen per schema; een `enum` heeft hoogstens 256 waarden.
- `required` noemt alleen eigen properties, elk één keer. `minItems`, `maxItems`, `minLength` en `maxLength` zijn gehele getallen van 0 of meer; `minimum` en `maximum` mogen elk getal zijn. Een ondergrens is nooit groter dan de bovengrens.
- Klopt een set niet, dan noemt de melding het pad van de eerste fout, bijvoorbeeld `categories[0].clues[2].question: gebruik 1–240 tekens.`

Sets bewaren

Alle routes hieronder vragen een accounttoken; een bevestigd e-mailadres is niet nodig. De set van een ander account bestaat voor jou niet: `404 set_not_found`. Bewaren, wijzigen en verwijderen tellen samen tot 30 per minuut per IP-adres.

Een **set** ziet er zo uit. `size` is de grootte van de inhoud in bytes, tijden zijn Unix-seconden. In een lijst ontbreekt `data`.

```
{
  "id": "5d0c7a1e-…",
  "game_id": "jeopardy",
  "kind": "board",
  "title": "Dani's filmavond",
  "data": {"categories": [
    {"title": "Films", "clues": [{"question": "…", "answer": "…"}, …]}, …
  ]},
  "size": 4210,
  "created_at": 1792660000,
  "updated_at": 1792663600
}
```

`GET` `/api/v1/accounts/sets`

Toegang accounttoken

Jouw sets zonder inhoud, laatst gewijzigd eerst: `{"sets": [Set, …]}`. Filter met `?game_id=jeopardy` en `?kind=board`.

`POST` `/api/v1/accounts/sets`

Toegang accounttoken

Bewaart een nieuwe set. De soort moet in de actieve versie van de game staan en de inhoud moet aan het schema voldoen. Ingebouwde games controleren daarnaast hun eigen regels. De titel is 1–80 tekens zonder controletekens.

Verzoek

```
{"game_id": "jeopardy", "kind": "board", "title": "Dani's filmavond",
 "data": {"categories": […]}}
```

Antwoord 201: de set, met `data`.

Fouten: `400 invalid_request` (titel, onbekende soort of inhoud), `400 invalid_json`, `401 unauthorized`, `404 game_not_found` (ook uitgezet of geblokkeerd), `409 quota_sets` (al 50 sets), `413 set_too_large` (meer dan 64 KiB), `429 rate_limit`

`GET` `/api/v1/accounts/sets/{id}`

Toegang accounttoken

Eén set, met `data`.

Fouten: `401 unauthorized`, `404 set_not_found`

`PUT /api/v1/accounts/sets/{id}`

Toegang accounttoken

Vervangt titel en inhoud: `{"title": "…", "data": {…}}`. Game en soort blijven gelijk. De engine controleert opnieuw tegen de actieve versie. Kamers die de set al gebruiken, houden hun kopie. Geeft de set terug.

Fouten: `400 invalid_request`, `401 unauthorized`, `404 set_not_found`, `404 game_not_found`, `413 set_too_large`, `429 rate_limit`

`DELETE /api/v1/accounts/sets/{id}`

Toegang accounttoken

Verwijdert de set. Antwoord `204`. Kamers die hem gebruiken, spelen door met hun kopie.

Fouten: `401 unauthorized`, `404 set_not_found`, `429 rate_limit`

Een kamer met een eigen set

Stuur `set_id` mee bij `POST /api/v1/rooms`, met het accounttoken van de eigenaar van de set.

```
{"game_id": "jeopardy", "set_id": "5d0c7a1e-…", "settings": {"team_count": 3}}
```

- De set hoort bij het ingelogde account en bij `game_id`, en de soort staat in de versie die de kamer krijgt. De engine controleert de inhoud opnieuw tegen die versie.
- De kamer krijgt een kopie. Later wijzigen of verwijderen verandert een bestaande kamer niet.
- Elke snapshot toont `"set": {"id": "5d0c7a1e-…", "kind": "board", "title": "Dani's filmavond"}`, zodat lobby en scherm de naam kunnen laten zien. De inhoud staat nooit in een snapshot.
- Overal waar de engine anders `data.json` gebruikt, gebruikt hij de set: een servergame zet er de ronde mee op, en `GET /api/v1/rooms/{code}/game-data` geeft de host de inhoud. Een eigen browsergame leest de gekozen set dus gewoon met `gameData(code, token)`.
- Fouten: `401 unauthorized` (geen of ongeldig accounttoken), `404 set_not_found`, `400 invalid_request` (set van een ander spel, of soort of inhoud past niet bij de versie), `409 release_changed` (net een andere versie actief geworden; probeer opnieuw).

Jeopardy: een eigen vragenbord

Jeopardy kent de soort `board`:

```
{"categories": [
  {"title": "Films", "clues": [
    {"question": "Welke film won in 2020 de Oscar voor beste film?", "answer": "Parasite"},
    {"question": "…", "answer": "…"}, {"question": "…", "answer": "…"},
    {"question": "…", "answer": "…"}, {"question": "…", "answer": "…"}
  ]}
]}
```

- 1–6 categorieën met elk precies 5 vragen. Een categorienaam is 1–40 tekens, een vraag en een antwoord 1–240 tekens, en geen tekst is leeg.

- Categorienamen zijn uniek, ongeacht hoofdletters.
- De waarde ligt vast per rij, van 100 tot 500. De engine maakt de vraag-ID's, zoals `c2-300` (categorie 2, 300 punten).
- Wie in een team een show op een eigen bord speelt, verdient `eigen-bord`.

Op `/home/sets` maak en bewerk je borden in een raster. Andere soorten krijgen sinds 0.5.3 een formulier als hun velden plat zijn: tekst, getallen, aan/uit, lijsten met één tekst per regel en lijsten van kleine items met een blok per item. Het formulier gebruikt `title` en `description` als label en uitleg, een `title` op `items` als naam van één item, en zet verplichte velden bovenaan in de volgorde van `required`. Diepere schema's krijgen een JSON-editor met de meldingen van de server. De pagina voor een nieuwe kamer biedt het standaardbord, je eigen borden en een link om er een te maken. Gasten zien daar een korte uitnodiging om in te loggen.

Ingebouwde serveracties

De regels van Common Dice en Jeopardy draaien op de server. Hun pagina's sturen acties en tekenen snapshots. De server kiest de uitkomst; animaties en realtime laten die alleen zien.

Common Dice

Instellingen: `dice_count` 1–20, `sides` 4, 6, 8, 10, 12 of 20, `roll_permission` `everyone` of `owner`, `theme` `ivory`, `obsidian` of `candy`. Standaard `{dice_count:5, sides:6, roll_permission:"everyone", theme:"ivory"}`.

Actie	Wie	Wat er gebeurt
`roll` `{held?: [0, 2]}`	Iedere deelnemer; bij `owner` alleen de host.	Alleen tijdens `running`. Gooit de vrije stenen opnieuw. `held` verwijst naar de huidige worp; minstens één steen blijft vrij.
`clear_history` `{}`	Host	Wist de zichtbare worpgeschiedenis.
`start` `{}`, `finish` `{}`	In `hostless` iedere deelnemer, anders de host.	Start of rondt de ronde af.

`roll_permission:"owner"` gaat niet samen met `hostless`. De publieke state bevat `results`, `roller`, `roller_id`, `sequence`, `configuration` (bijvoorbeeld `"5d6"`), `held`, `rolled_at` in epochmilliseconden en hoogstens twintig items in `history`.

Jeopardy

Instelling: `team_count` 2–6, standaard 3. Alleen `moderator`; geen late joining. `team_count` kan alleen in de lobby veranderen, zolang nog niemand een team koos.

Actie	Wie	Wat er gebeurt
`choose_team` `{team_id}`	Speler, in de lobby	Kiest een team. Starten vereist dat iedere speler er een heeft.
`buzz` `{}`	Speler	De eerste geldige buzzer bij een open vraag wint. Anders `409 buzzer_closed`.
`select_clue` `{clue_id}`	Host	Opent een vraag. Het antwoord staat in `host_state.active_answer`.
`judge` `{correct}`	Host	Kent de score één keer toe; bij fout gaat de vraag open voor de andere teams.
`reveal`, `reopen`, `close_clue`, `finish`	Host	Onthult het antwoord, opent opnieuw, sluit de vraag of rondt het spel af.

Lege teams kunnen niet winnen. Ongeldige zetten geven `409 invalid_game_action`.

Met een eigen vragenbord (`set_id`, soort `board`) speelt de kamer op dat bord in plaats van op `data.json`; zie *Eigen sets*. Spelers sturen bij het drukken ook het realtime-event `pressed`, zodat host en scherm meteen tonen wie drukte en hoeveel milliseconden later. `buzz` blijft de actie die telt.

SDK-client

`@casb02/mini-engine-sdk` versie 0.5.2: een getypeerde client voor browsers en Node.js 22 of nieuwer. Het pakket staat op GitHub Packages; de broncode staat openbaar op GitHub (`Casb02/mini-engine-sdk`, MIT).

Nieuw in 0.5.0 voor engine 0.5: `client.realtime`, `client.award`, de events `throttled` en `achievement`, de setmethoden en `achievements()` op `ApiClient`, `set_id` bij het maken van een kamer, `set` in de snapshot, en `achievements` en `set_kinds` in het manifest. Nieuw in 0.5.1: `icon` in `Manifest` en `host_player_id` in `Snapshot`. Nieuw in 0.5.2: `'tv'` in `HostMode`. De server en `me dev` serveren deze versie onder `/sdk/`. De SDK heeft geen methoden voor `/api/v1/dev`; daarvoor is de `me`-CLI.

Laden

In een gamepagina importeer je de build die de server zelf serveert. Daarbuiten installeer je het pakket van GitHub Packages. Zet daarvoor eenmalig de scope in `.npmrc`; GitHub vraagt ook voor een openbaar pakket een token met `read:packages`.

```
@casb02:registry=https://npm.pkg.github.com
//npm.pkg.github.com/:_authToken=${GITHUB_TOKEN}
```

Daarna: `npm install @casb02/mini-engine-sdk`. Zonder token kan het rechtstreeks uit de repository: `npm install github:Casb02/mini-engine-sdk#v0.5.2`.

```
import { ApiClient, MiniEngine, loadSession } from '/sdk/mini-engine.js';

const code = new URL(location.href).searchParams.get('room') || '';
const credentials = loadSession(code, 'player');
if (!credentials) throw new Error('Open deze pagina via de kamer.');

const client = new MiniEngine(credentials).connect();
client.on('snapshot', room => render(room));
client.on('error', error => showError(error.message));
await client.waitForSnapshot();
await client.ready(true);
// Zodra het spel loopt:
await client.action('roll', { held: [] });
```

ApiClient

`new ApiClient({ baseUrl?, accountToken?, fetch? })`. Zonder `baseUrl` gebruikt hij de huidige origin. Methoden geven een Promise en gooien een `MiniEngineError` bij een HTTP-fout.

Methode	Route	Geeft
`getGames()`	`GET /games`	`{ games: Manifest[] }`
`getGame(id)`	`GET /games/{id}`	`Manifest`
`createRoom(options)`	`POST /rooms`	`RoomCredentials`

Methode	Route	Geeft
`joinRoom(code, name)`	`POST /rooms/{code}/join`	`RoomCredentials`
`roomInfo(code)`	`GET /rooms/{code}/info`	openbare kamerinfo
`getRoom(code, token)`	`GET /rooms/{code}`	`Snapshot`
`gameData(code, token)`	`GET /rooms/{code}/game-data`	de private speldata
`displayToken(code, token)`	`POST /rooms/{code}/display-token`	`{ token }`
`accountConfig()`	`GET /accounts/config`	`{ turnstile_site_key }`
`register(input, { turnstileToken? })`	`POST /accounts/register`	`AccountSession`; `input` met `email`
`verifyEmail(token)`	`POST /accounts/verify-email`	`{ account }`; met `accountToken`
`setEmail(email, { turnstileToken? })`	`PUT /accounts/email`	`{ account, verification_sent }`
`login(input)`	`POST /accounts/login`	`AccountSession`
`me()`	`GET /accounts/me`	`Account`
`stats()`	`GET /accounts/stats`	`AccountStats`
`achievements(accountToken?)`	`GET /accounts/achievements`	`{ achievements: AccountAchievement[] }`
`listSets({ game_id?, kind? })`	`GET /accounts/sets`	`{ sets: SetSummary[] }`
`getSet(id)`	`GET /accounts/sets/{id}`	`SavedSet`
`createSet(input)`	`POST /accounts/sets`	`SavedSet`; `input` is een `NewSet`
`updateSet(id, { title, data })`	`PUT /accounts/sets/{id}`	`SavedSet`
`deleteSet(id)`	`DELETE /accounts/sets/{id}`	`null`
`logout()`	`POST /accounts/logout`	`null`; wist `accountToken`

`createRoom` en `joinRoom` sturen het `accountToken` mee als dat is ingesteld; `createRoom` accepteert `set_id`. De accountmethoden gebruiken het `accountToken`; `achievements` kan ook een ander token krijgen. `register` en `login` bewaren het token niet; zet het zelf in `accountToken`. `verifyEmail` werkt alleen met het `accountToken` van het account waarvoor de link is. Losse helpers voor de huidige origin: `getGames()`, `createRoom(options)`, `joinRoom(code, name)` en `http(path, options)`, die JSON ophaalt met een optioneel `token`.

Vraagt de server om een robotcheck (`accountConfig()` geeft dan een sitekey), geef `register` en `setEmail` dan het token van het widget mee als `{ turnstileToken }`; de SDK zet het in de header `X-Turnstile-Token`.

MiniEngine

```
new MiniEngine({ code, token, baseUrl?, reconnectDelay?, requestTimeout?,
                 WebSocket?, realtime? })
```

Een `RoomCredentials`-object past direct. Standaard: `reconnectDelay` 500 ms, verdubbelend tot hoogstens 15 seconden met wat spreiding; `requestTimeout` 10 seconden; `realtime: { setInterval: 33 }`, het minimum aantal milliseconden tussen twee keer waarden versturen.

Lid	Wat het doet
`connect()`	Opent de verbinding en authenticeert. Geeft de client terug.
`resume()`	Verbindt opnieuw na een stop, bijvoorbeeld na `session_replaced`. De laatste snapshot blijft staan tot er een nieuwe is.
`close()`	Stopt, wijst open opdrachten af en verbindt niet opnieuw.
`on(type, fn)`, `once(type, fn)`	Luistert naar een event. Geeft een functie terug die het luisteren stopt.
`waitForSnapshot(predicate?, { timeout?, signal? })`	Wacht op een snapshot die aan de voorwaarde voldoet.
`action(name, data?, { actionId? })`	Stuurt een actie, standaard met een nieuwe unieke `action_id`.
`ready(ready = true)`	Zet readiness.
`configure(config)`	Past `display_enabled` of `allow_late_join` aan met de huidige versie.
`setSettings(settings)`	Vervangt alle instellingen met de huidige versie.
`setState(update)`	Publiceert de volledige state; alleen voor een hostruntime.
`lifecycle(status)`	Zet de status door.
`award(playerId, achievementId)`	Kent een achievement toe; alleen voor de host van een hostruntime. Lost ook op als de speler het al had.
`request(type, payload)`	Stuurt een willekeurig bericht met een eigen `request_id` en wacht op de `ack`.
`realtime`	Events en waarden voor deze kamer; zie hieronder.
`room`, `connected`, `code`, `token`	De laatste snapshot, de verbindingsstatus en de credentials.

Alle opdrachten geven een Promise die oplost met de `ack`. Ze worden afgewezen bij een `error`, bij een time-out (`request_timeout`), bij verbreken (`disconnected`) of als er nog geen verbinding is (`not_connected`).

Events

Event	Waarde
`snapshot`	De nieuwe `Snapshot`. De eerste snapshot markeert de client als verbonden.
`action`	Een `ActionEnvelope`; alleen voor de host van een hostruntime.
`status`	`connecting`, `connected`, `reconnecting` of `disconnected`.
`error`	Een `MiniEngineError` met `code` en `message`.
`closed`	De sluitreden als de server definitief sloot.
`throttled`	`{ dropped }`: de server gooide zoveel realtimeberichten van deze verbinding weg.
`achievement`	`{ player_id, achievement }`: een speler verdiende een achievement voor het eerst.

client.realtime

Lid	Wat het doet
`emit(channel, data?, { to?, echo? })`	Stuurt een event. Geeft `false` zonder verbinding; het event wordt dan niet bewaard of later verstuurd.
`on(channel, fn)`	Luistert naar één kanaal, of naar alle met `'*'`: `fn(data, from, ts, channel)`. Geeft een functie terug die het luisteren stopt.
`set(key, value)`	Zet een eigen waarde; `null` wist hem. Per sleutel samengevoegd en hoogstens eens per `setInterval` verstuurd. Zonder verbinding gebeurt er niets.
`value(from, key)`	De laatste waarde van een afzender: `'host'`, `'display'`, een speler-ID of een `from`-object.
`onValue(key, fn)`	Luistert naar één sleutel, of naar alle met `'*'`: `fn(value, from, ts, key)`, met `value` `null` bij wissen.
`reset()`	Alleen de host: wist alle waarden in de kamer. Waarden die je daarna zet, blijven staan.

Je eigen waarden staan meteen in `value()`, ook voordat de server ze heeft. Na opnieuw verbinden bouwt de client de waarden opnieuw op uit `rt.values`; waarden die nog niet waren verstuurd, vervallen. `close()` wist alles. Het vierde argument van de handlers (`channel` of `key`) is handig bij `'*'`.

Sessies

Functie	Wat het doet
`sessionKey(code, role)`	Geeft `mini-engine:CODE:ROLE`.
`saveSession(credentials, storage?)`	Bewaart credentials, standaard in `sessionStorage`.
`loadSession(code, role, storage?)`	Leest ze terug, of `null` als ze ontbreken of niet kloppen.
`removeSession(code, role, storage?)`	Verwijdert ze.

Fouten en typen

`MiniEngineError` breidt `Error` uit met `code` en, bij HTTP, `status`. Bij serverfouten is `code` de foutcode van de server; daarnaast bestaan clientcodes als `not_connected`, `request_timeout`, `snapshot_timeout`, `disconnected`, `closed` en `missing_base_url`.

De belangrijkste typen: `Json`, `Role`, `HostMode`, `Lifecycle`, `ConnectionStatus`, `RoomConfig`, `Player`, `RoomSet`, `Snapshot<PublicState, Settings, PrivateState>`, `Manifest`, `SettingsProperty`, `AchievementDefinition`, `SetKind`, `SetSchema`, `RoomCredentials`, `CreateRoomOptions`, `Account`, `AccountSession`, `AccountStats`, `AccountAchievement`, `AchievementSource`, `AchievementEarned`, `SetSummary`, `SavedSet`, `NewSet`, `SetFilter`, `Realtime`, `RealtimeFrom`, `RealtimeTarget`, `RealtimeEmitOptions`, `RealtimeThrottled`, `Ack`, `ActionEnvelope`, `StateUpdate` en `ActionPayloads` voor de ingebouwde acties. `MiniEngine<P, S, V>` geeft de typen van publieke state, instellingen en private state door aan `room` en de events.

Een eigen browserhost

Bij `runtime:"host"` verwerkt de hostpagina de acties en publiceert de hele volgende state. Valideer speler, payload, status en spelregels; vertrouw nooit een identiteit of score uit de clientdata. Zet alle mutaties in één wachtrij en wacht op snapshot en ack voordat de volgende een versie gebruikt.

```
client.on('action', async ({ player_id, name, data }) => {
  if (name !== 'vote' || client.room?.status !== 'running') return;
  const next = tally(client.room.public_state, player_id, data);
  await client.setState({
    expected_version: client.room.version,
    public_state: next,
  });
});
```

Speelt de host mee, dan stuurt zijn pagina geen actie voor de eigen zet. Ze past die zet direct toe op de state, met `client.room.host_player_id` als speler, en publiceert hem met `setState`. Een achievement toekennen gaat met `await client.award(player_id, 'snelste')`; zie *Achievements* voor de regels. De scaffolder maakt een werkend voorbeeld: `me game new <slug>` uit de CLI, of `create-mini-game <slug> [--directory pad]` uit de SDK.

Manifest en instellingen

Een game is een map met een manifest, drie rolpagina's en publieke assets. Bij een upload is het id in de URL de game-ID; het moet gelijk zijn aan `id` in het manifest.

```
my-game/
  manifest.json
  host.html
  player.html
  display.html     verplicht bij has_display: true
  assets/          publiek, onder /games/my-game/assets/…
  data.json        optioneel en privé; alleen de host leest dit
```

Manifestvelden

Veld	Type	Betekenis
`id`	string	De game-ID: 1–48 kleine letters, cijfers of `-`.
`title`, `description`, `version`	string	Naam, korte omschrijving en eigen versie van de game. Bij een upload is `version` de naam van de release.
`min_players`, `max_players`	getal	Grenzen voor `max_players` van een kamer.
`has_display`	boolean	Of er een gedeeld scherm is; dan is `display.html` verplicht.
`default_settings`	object	Standaardinstellingen; moeten aan het schema voldoen.
`settings_schema`	object	Het instellingenschema, zie hieronder.
`runtime`	`host` | `server`	Standaard `host`. Een upload moet `host` zijn.
`host_modes`	lijst	Toegestane hostmodi. Standaard `["moderator"]`, zodat oudere packages hetzelfde blijven. Een browsergame kiest uit `playing`, `tv` en `moderator`; een gezelschapsspel meestal `["playing", "tv"]`.
`default_host_mode`	string	Moet in `host_modes` staan. Standaard `moderator`; voor de meeste nieuwe games `playing`.
`supports_late_join`	boolean	Standaard `true` voor oudere manifests. Kies het expliciet voor nieuwe games.
`achievements`	lijst	Optioneel, hoogstens 50: `id`, `title`, `description`. Zie *Achievements*.
`set_kinds`	lijst	Optioneel, hoogstens 4 soorten sets: `id`, `title`, `description`, `schema`. Zie *Eigen sets*.
`icon`	string	Optioneel, sinds 0.5.1: het pad van een SVG, `assets/<naam>.svg`. Zie hieronder.

```
{
  "id": "my-game",
  "title": "My Game",
  "description": "Een spel voor vrienden.",
  "version": "1.0.0",
  "runtime": "host",
  "host_modes": ["playing", "tv"],
  "default_host_mode": "playing",
  "supports_late_join": false,
  "min_players": 1,
  "max_players": 12,
  "has_display": true,
  "default_settings": {"rounds": 5, "mode": "normal"},
  "settings_schema": {
    "type": "object",
    "properties": {
      "rounds": {"type": "integer", "minimum": 1, "maximum": 20, "title": "Rondes"},
      "mode": {"type": "string", "enum": ["normal", "quick"]}
    },
    "required": ["rounds", "mode"],
    "additionalProperties": false
  },
  "achievements": [
    {"id": "snelste", "title": "Snelste", "description": "Geef het snelste goede antwoord."}
  ],
  "icon": "assets/icon.svg"
}
```

Icoon

Zonder `icon` toont de startpagina een tegel met de eerste letter van de titel. Met `icon` controleert de server bij een upload, en `me game upload` en `me dev` al op je eigen computer:

- het pad begint met `assets/`, eindigt op `.svg` en het bestand zit in het package;
- het is hoogstens 64 KiB, UTF-8 en bevat `<svg`;
- het bevat geen `<script`, `<foreignObject`, `javascript:` of een `on…=`-attribuut, ongeacht hoofdletters.

Anders weigert de server het package met `Invalid icon: …`. Maak het icoon vierkant (`viewBox="0 0 48 48"`) als een volle tegel met een eigen achtergrond (`rx="12"`) en een of twee eenvoudige vormen in de kleuren van de game. Geen tekst, geen lettertypes en geen verwijzingen naar andere bestanden. Houd de rand van de tegel effen: de spelkaart neemt die kleur over als vlak.

Instellingenschema

Een klein deel van JSON Schema. De server controleert schema en standaardwaarden bij een upload en weigert alles wat hij niet kent; `me dev` en `me game upload` doen dat al op je eigen computer. Het formulier voor een nieuwe kamer toont de instellingen in de volgorde van `required`, daarna de rest.

- Het schema is een `object` met `properties`, optioneel `required` en verplicht `additionalProperties:false`. Annotaties `title`, `description` en `default` mogen.
- Een property heeft `type` `integer`, `number`, `string` of `boolean`, en optioneel `enum` (hoogstens 64 unieke waarden), `minimum`, `maximum`, `maxLength`, `title`, `description` en `default`.

- Arrays, geneste objecten en `$ref` worden niet ondersteund. Voor rijkere inhoud zijn er eigen sets, met het set-schema; zie *Eigen sets*.

De spelregels kiest de maker in het formulier waarmee hij de kamer maakt. Daarna liggen ze vast: een game toont ze als gewone tekst, bijvoorbeeld een korte regel "Regels: …", en biedt geen knoppen om ze te veranderen. `setSettings` blijft bestaan voor een game die het echt nodig heeft, maar regels veranderen terwijl er gespeeld wordt, is oneerlijk, en een formulier dat bij elke snapshot opnieuw tekent, verliest makkelijk een klik.

Gebruik absolute paden in de HTML, bijvoorbeeld `/games/my-game/assets/app.js` en `/sdk/mini-engine.js`. Bij een geüploade game zet de server `/games/my-game/assets/` in de HTML van een kamer om naar de vaste versie. Imports tussen je eigen JS-modules schrijf je relatief (`./logic.js`), want daar verandert de server niets aan.

Grenzen en rate limits

Vaste grenzen in 0.6. Overschrijden geeft een fout; niets wordt stilletjes ingekort. Alleen realtime gooit berichten weg in plaats van te weigeren, en meldt dat met `rt.throttled`.

Wat	Grens	Bij overschrijden
HTTP-body	96 KiB	`413 invalid_json`
Uploadbody (`POST …/versions`)	16 MiB	`413 invalid_json`
Websocketbericht	96 KiB	verbinding sluit
Gepubliceerde state (publiek, host en spelers samen)	64 KiB	`413 state_too_large`
Actiedata en instellingen	8 KiB	`400 invalid_request`
Actienaam	1–64 tekens: letters, cijfers, `_ . -`	`400 invalid_request`
`action_id` , `request_id`	hoogstens 64 tekens	`400 invalid_request`
Deduplicatievenster	laatste 256 action-ID's per kamer	oudere ID's tellen weer als nieuw
Spelersnaam, weergavenaam	1–24 tekens	`400 invalid_request`
Standaard `max_players`	maximum van de game, hoogstens 24	hoger mag tot het gamemaximum
Kamers op de server	500	`503 room_limit`
Websocketverbindingen	1024	`503 connection_limit`
Kamer zonder activiteit	12 uur	sluit met `room_expired`
Websocket: auth en stilte	auth binnen 5 s; hoogstens 45 s stil (ping elke 15 s)	`auth_timeout` , `heartbeat_timeout`
Realtime: `data` of `value`	2 KiB geserialiseerd	`error` met `invalid_request`
Realtime: kanaal en sleutel	1–64 tekens: letters, cijfers, `_ . : -`	`error` met `invalid_request`
Realtime: ontvangers in `to`	hoogstens 16 speler-ID's	`error` met `invalid_request`
Realtimeberichten	per verbinding 60 per seconde, buffer 120	weggegooid; `rt.throttled` hoogstens eens per 5 s
Realtimewaarden	64 sleutels per afzender, 256 KiB per kamer	`rt.set` weggegooid; `rt.throttled`
Accountsessie	30 dagen	`401 unauthorized`
E-mailadres	hoogstens 254 tekens	`400 invalid_request`
Bevestigingslink	48 uur, één keer	`400 invalid_token`
Bevestigingsmails	één per account per minuut	`429 rate_limit`
Achievements in een manifest	50 per game; id 1–48 tekens, titel 1–60, beschrijving 1–160	bij een upload `400 invalid_request`

Wat	Grens	Bij overschrijden
Achievements toekennen door de host	30 per kamer per minuut	`429 rate_limit`
Soorten sets in een manifest	4 per game; schema hoogstens 8 niveaus, 256 onderdelen en 256 `enum`-waarden	bij een upload `400 invalid_request`
Set	titel 1–80 tekens; inhoud hoogstens 64 KiB	`400 invalid_request`, `413 set_too_large`
Sets per account	50	`409 quota_sets`
Icoon (`icon`)	SVG onder `assets/`, hoogstens 64 KiB, zonder scripts	bij een upload `400 invalid_request`
`enum` in het instellingenschema	hoogstens 64 unieke waarden	bij een upload `400 invalid_request`
Gamepackage	2 MiB per bestand, 10 MiB en 4096 bestanden per game	bij een upload `400 invalid_request`
Geüploade games	10 per account	`409 quota_games`
Bewaarde versies	10 per game	`409 quota_releases`
Versienaam van een upload	1–64 tekens: letters, cijfers, `. + _ -`	`400 invalid_request`
Bestandspad in een upload	hoogstens 255 tekens en 32 delen	`400 invalid_request`

Rate limits

HTTP-tellers lopen per IP-adres in vaste vensters van zestig seconden, of tien minuten waar dat staat. IPv6-adressen tellen per /64-blok. Daarnaast wacht een gebruikersnaam een kwartier na tien mislukte inlogpogingen, van welk adres ook; een geslaagde login zet de teller terug. De SDK wacht na `rate_limit` minstens 1,2 seconden voordat hij opnieuw verbindt.

Wat	Limiet	Fout
`POST /api/v1/rooms` en `POST …/join`	60 per minuut per IP-adres	`429 rate_limit`
`POST /api/v1/accounts/register`, `…/login` en `…/verify-email`	10 per minuut per IP-adres	`429 rate_limit`
Mails: `POST …/register` en `PUT /api/v1/accounts/email`	5 per tien minuten per IP-adres	`429 rate_limit`
`POST`, `PUT`, `DELETE` onder `/api/v1/accounts/sets`	30 per minuut per IP-adres	`429 rate_limit`
`POST`, `PUT`, `DELETE` onder `/api/v1/dev/`	60 per minuut per IP-adres	`429 rate_limit`
Uploads (`POST …/versions`)	20 per tien minuten per IP-adres	`429 rate_limit`
Verkeerde dev-key	na 10 keer 401 binnen tien minuten: tien minuten geweigerd, ook met de goede sleutel	`429 rate_limit`
Websocketberichten	per verbinding 240 per seconde voor de host, 60 voor andere rollen; realtime telt apart	`error` en sluiten met `rate_limit`

Foutformaat en foutcodes

Elke fout heeft dezelfde vorm, over HTTP en over de websocket. Reageer op de `code`; toon de `message` aan mensen.

```
{"error": {"code": "room_full", "message": "Deze room is vol."}}
```

Over de websocket staat dezelfde fout in `{"type":"error","request_id":"…","error":{…}}`.

Codes

Status	Code	Wanneer
400	`invalid_request`	Een veld klopt niet: naam, e-mailadres, instellingen, config, actie, payload, realtimebericht of set.
400	`invalid_token`	Bevestigingslink onbekend, verlopen, al gebruikt of voor een ander account.
400	`invalid_json`	Geen geldige JSON, onbekende velden of verkeerd `Content-Type`. Bij een te grote body met status 413.
400	`turnstile_required`, `turnstile_failed`	Robotcheck: het token in `X-Turnstile-Token` ontbreekt, of Cloudflare keurde het af.
401	`unauthorized`, `auth_timeout`	Token ontbreekt, is ongeldig, ingetrokken of verlopen; of geen websocket-auth binnen vijf seconden.
403	`forbidden`, `origin_forbidden`	Deze rol mag dit niet (ook `rt.reset` door een andere rol dan de host, en toekennen in een servergame), de game is van een andere ontwikkelaar, of de browser-origin is niet toegestaan.
403	`not_eligible`	Toekennen kan niet: de speler is een gast, de kamer is zonder account gemaakt, of de ontvanger is het account van de maker en niet de stoel van de meespelende host.
403	`game_blocked`	De beheerder heeft deze game geblokkeerd; de ontwikkelaar kan hem niet aanzetten.
404	`room_not_found`	De kamer bestaat niet of is verlopen.
404	`set_not_found`	De set bestaat niet of is van een ander account.
404	`achievement_unknown`	De versie van de kamer declareert dit achievement niet.
404	`game_not_found`	De game bestaat niet, is uitgezet (zonder kamer) of is geblokkeerd. Onder `/api/v1/dev` en `/api/v1/accounts/games` ook: niet van jou.
404	`game_page_not_found`, `asset_not_found`, `player_not_found`, `not_found`	Rolpagina, asset, speler of route bestaat niet.
404	`release_not_found`	Deze versie van de game bestaat niet (meer).
405	`method_not_allowed`	De route bestaat, de methode niet.
409	`version_conflict`	De versie is intussen veranderd. Lees de nieuwste snapshot.

Status	Code	Wanneer
409	`room_started`, `room_full`, `name_taken`, `already_joined`, `username_taken`	Binnenkomen, klaarzetten of configureren kan niet (meer), de naam of gebruikersnaam is bezet, of dit account heeft al een stoel.
409	`email_taken`, `email_already_verified`	Het adres is al bevestigd door een ander account, of al bevestigd voor dit account.
409	`invalid_transition`, `players_not_ready`	Lifecycle in de verkeerde volgorde, of niet iedereen staat klaar.
409	`game_not_running`, `host_offline`	Een spelactie buiten `running`, een achievement toekennen in de lobby, of een actie bij een hostruntime zonder verbonden host.
409	`session_replaced`	Een nieuwere verbinding gebruikt dit token.
409	`invalid_game_action`, `buzzer_closed`, `teams_locked`, `teams_required`	Spelregels van een servergame.
409	`game_id_taken`, `version_exists`	Upload met het id van een ingebouwde game, of een versie die al bestaat.
409	`quota_games`, `quota_releases`	Al 10 games op dit account, of al 10 versies van deze game.
409	`release_active`, `release_in_use`, `game_in_use`	Verwijderen kan niet: de actieve versie, of een versie of game die een lopende kamer nog speelt.
409	`quota_sets`	Al 50 sets op dit account. Verwijder er eerst een.
409	`release_changed`	Tijdens het maken van een kamer met een eigen set werd een andere versie actief. Probeer het opnieuw.
413	`state_too_large`, `set_too_large`	Gepubliceerde state of de inhoud van een set groter dan 64 KiB.
429	`rate_limit`	Te veel verzoeken, berichten, mails of toekenningen.
500	`internal_error`	Opslaan mislukte. De oude state blijft staan.
503	`room_limit`, `connection_limit`	De server zit vol. Probeer het later opnieuw.
503	`turnstile_unavailable`	Cloudflare Turnstile was niet bereikbaar. Probeer het zo opnieuw.